

Real-Time Incident Management Dashboard

Final Report for the Distributed Systems Course 2025/2026

Alessandro Martini

`alessandro.martini10@studio.unibo.it`

October 4, 2026

This project presents a distributed real-time incident management system designed for collaborative ticket resolution in operational environments. The system enables multiple operators to simultaneously interact with a shared pool of incidents through a web-based Kanban dashboard. The architecture follows a three-tier pattern: a Vue.js 3 single-page application serves as the presentation layer, an Express.js API with Socket.IO handles real-time communication and business logic, while a MongoDB Replica Set ensures persistent strongly-consistent state and Redis provides distributed coordination. Key distributed systems patterns implemented include: (i) Redis-based distributed locking with Lua scripts for atomic claim/release operations, preventing race conditions during ticket assignment; (ii) optimistic locking via document versioning in MongoDB to avoid lost updates during concurrent modifications; (iii) leader election using Redis TTL keys for exclusive execution of background tasks (automatic escalation of stale incidents); (iv) MongoDB Change Streams for reactive, event-driven synchronization across all connected clients without polling. The entire system is containerized and orchestrated via Kubernetes, featuring a 3-node MongoDB StatefulSet for data persistence and failover, Horizontal Pod Autoscaler for the API tier (3–10 replicas), and an Ingress with session affinity.

1 Concept

1.1 Product Type

The project consists of a web application with a real-time graphical user interface (GUI). Specifically, it is a Single-Page Application (SPA) built with Vue.js 3, served as a static web application and interacting with a distributed backend via HTTP and WebSocket protocols. The frontend presents a Kanban-style dashboard that allows multiple operators to visualize, create, claim, and resolve incidents in real time.

1.2 Use Case Description

1.2.1 What the software does

The system implements a **real-time incident management board** (similar to a ticketing or ITSM system) with the following core capabilities:

- **Incident registration:** operators can create new incidents with a title and initial status `open`
- **Ticket claiming:** an operator can *lock* an incident for exclusive processing, preventing other operators from simultaneously working on the same ticket (mutual exclusion)
- **Ticket resolution:** the claiming operator can mark an incident as `closed`, releasing the lock and notifying all connected clients
- **Automatic escalation:** a background task periodically scans for incidents that have remained `open` for more than 5 minutes and automatically escalates them to `escalated` status, signaling SLA breach
- **Real-time synchronization:** all connected clients receive live updates whenever an incident is created, modified, locked, unlocked, or escalated, without manual page refresh
- **User presence tracking:** the system displays the list of currently connected operators

Redis is used for **transient coordination state** (not persistent data):

- Distributed locks for ticket claiming (keys with TTL)
- Leader election key for the background escalation task
- Socket.IO adapter for cross-pod message broadcasting

1.3 Why Distribution is Needed

1.3.1 Fault Tolerance

The system must remain operational even if individual components fail. With a single-node deployment, the failure of the API server or the database would render the entire ticketing system unusable—a critical risk in operational environments where incident tracking cannot afford downtime. The Kubernetes architecture addresses this by:

- Running **3 API replicas** minimum, so the loss of one pod does not interrupt service
- Deploying MongoDB as a **3-node Replica Set**, which tolerates the failure of one node while maintaining a primary for writes
- Using Redis for shared state, so any API pod can take over coordination tasks

1.3.2 Scalability

As the number of concurrent operators and incidents grows, a single API instance would become a bottleneck. The system employs a **Horizontal Pod Autoscaler** (HPA) that scales the API tier from 3 to 10 replicas based on CPU utilization, ensuring that:

- HTTP request handling capacity increases with load
- Socket.IO connections are distributed across multiple pods (via the Redis adapter)
- The background escalation task is executed by exactly one leader, regardless of pod count

1.3.3 Resource Sharing

Multiple operators must access and modify the **same shared pool of incidents** concurrently. Without distribution, the system would be limited to a single operator at a time, defeating its collaborative purpose. The distributed architecture enables:

- **Concurrent read access:** all operators see the same live state of the Kanban board
- **Coordinated write access:** distributed locks ensure that only one operator can claim a ticket at a time, while optimistic locking prevents conflicting updates

1.3.4 Geographic Distribution (Potential)

While the current deployment targets a single Kubernetes cluster (e.g., Minikube or a single cloud region), the architecture is designed to support geographic distribution. The MongoDB Replica Set could be extended with nodes in different data centers, and the API tier could be replicated across regions behind a global load balancer, enabling operators in different locations to collaborate on the same incident pool.

2 Requirements Elicitation and Analysis

2.1 Functional Requirements

FR1 Board access. An operator can join the board by registering a username/password pair and, on subsequent visits, logging in with those credentials.

Acceptance criterion: submitting valid credentials on the login form transitions the client from the login screen to the dashboard and establishes a persistent, authenticated WebSocket connection; submitting invalid credentials is rejected with an error and no connection is established.

FR2 Incident creation. An operator can create a new incident by providing a title; the system assigns it a default status of **open**, a creation timestamp, and the creator's identity.

Acceptance criterion: a valid `POST /api/incidents` request returns 201 and the new incident appears as a card in the “Open” column of every connected client without any client issuing a manual refresh.

FR3 Real-time board synchronization. All connected operators must observe incident creations, claims, releases, resolutions, and escalations as they happen.

Acceptance criterion: an action performed by operator A becomes visible on operator B’s board without B reloading the page or polling.

FR4 Ticket claiming (mutual exclusion). An operator can claim an `open` or `escalated` incident for exclusive handling; once claimed, no other operator can claim or resolve it.

Acceptance criterion: of two concurrent claim requests on the same incident, exactly one succeeds (200) and the other is rejected (409, reporting the current owner).

FR5 Explicit claim release. An operator holding a claim can release it without resolving the incident, returning it to the shared pool for other operators.

Acceptance criterion: after a successful release, the incident becomes claimable again immediately, without waiting for the automatic lock expiry.

FR6 Ticket resolution. The operator currently holding the claim on an incident can mark it as `closed`, recording who closed it and releasing the associated lock.

Acceptance criterion: a resolved incident moves to the “Resolved” column on every client and can no longer be claimed or resolved again.

FR7 Automatic SLA escalation. Incidents left in the `open` status beyond a configured threshold (5 minutes) are automatically transitioned to `escalated` without operator intervention.

Acceptance criterion: an incident older than the threshold changes to `escalated` within one execution cycle of the background task (≤ 10 seconds after threshold crossing) and moves to the “Escalated” column on every client.

FR8 Presence awareness. The system displays the set of operators currently connected to the board.

Acceptance criterion: the online-users list reflects a new connection within one broadcast cycle, and a user disappears from the list once their connection is terminated (including ungraceful terminations).

FR9 Filtering and search. An operator can narrow the visible incidents via free-text search on the title, an “escalated only” toggle, and a “mine only” toggle (incidents they created, claimed, or resolved).

Acceptance criterion: activating any filter combination updates only the Kanban columns; the KPI summary counters continue to reflect the true, unfiltered system state.

FR10 Event notifications. Operators receive non-blocking toast notifications for relevant events they did not personally trigger, and for local operation failures.
Acceptance criterion: every failed claim, release, or resolve action produces a visible notification describing the reason for failure; no user-initiated action fails silently.

2.2 Non-Functional Requirements

NFR1 No lost updates. Concurrent modification attempts on the same incident must never silently overwrite one another.

Realized by: optimistic locking via a `version` field, matched on every `PATCH`.

Acceptance criterion: a `PATCH` carrying a stale version is rejected with `409` and leaves the document unmodified.

NFR2 Server-enforced mutual exclusion. The claim mechanism must be enforced by the server, not merely suggested by the client interface.

Acceptance criterion: a resolution request issued by an operator who does not currently hold the incident's lock is rejected with `409`, even when the request bypasses the UI entirely (e.g., sent directly to the API).

NFR3 Fault tolerance of the API tier. The loss of any single API pod must not interrupt service for connected or new operators.

Acceptance criterion: deleting one of the ≥ 3 running `api` pods produces no visible downtime; remaining pods continue serving requests, and `Socket.IO` clients transparently reconnect to a healthy pod.

NFR4 Fault tolerance of the data tier. The loss of any single MongoDB node must not interrupt read/write availability nor lose committed data.

Realized by: a 3-node Replica Set with automatic primary election, protected by a `PodDisruptionBudget` (`minAvailable: 2`) that prevents voluntary evictions from breaking quorum.

Acceptance criterion: terminating one of the 3 `mongo` pods triggers automatic recovery (re-election if the primary was lost) without manual intervention, and the API resumes normal operation once the probe reports readiness again.

NFR5 Exactly-once background execution. The automatic escalation task must execute exactly once per cycle across the whole cluster, regardless of how many API pods are running.

Realized by: Redis-based leader election with an atomically-renewed lock (Lua script, preventing the split-brain window of a naive `GET+PEXPIRE`).

Acceptance criterion: with $N \geq 1$ running `api` pods, a given incident is escalated exactly once (no duplicate version increments, no duplicate log entries).

NFR6 Horizontal scalability. The API tier must absorb increased load by adding replicas automatically, without manual intervention.

Realized by: a `HorizontalPodAutoscaler` (3–10 replicas, 80% CPU utilization target).

Acceptance criterion: sustained CPU load above the threshold results in additional `api` pods being scheduled within the HPA’s stabilization window.

NFR7 Reconnection consistency. A client that experiences a temporary disconnection (e.g., during a rolling update of the API tier) must never regress to a state older than what it displayed before disconnecting, and must recover the *whole* observable state of each incident, including who currently holds its claim.

Realized by: a resynchronization-generation counter that discards any hydration response superseded by a later one, and a board snapshot that carries the claim state read from Redis rather than the MongoDB documents alone.

Acceptance criterion: rapid, repeated reconnects never leave the client displaying data fetched by an earlier, superseded hydration; and an operator who reloads the page while holding a claim still sees that claim, and retains the actions it authorizes.

NFR8 Self-healing presence data. The count of online operators must never permanently overcount stale sessions after an ungraceful pod termination.

Realized by: deriving presence from `io.fetchSockets()` (live socket introspection across the Redis adapter) instead of a manually-maintained Redis hash, which was found during development to leak stale entries on unclean shutdowns.

Acceptance criterion: after force-terminating an API pod, the online-user count converges to the correct value without manual state cleanup.

NFR9 UI eventual consistency. A given incident must never be rendered in two mutually-exclusive Kanban columns at once, regardless of the arrival order of independent real-time events describing the same state change.

Acceptance criterion: resolving a claimed incident never produces a transient duplicate card appearing simultaneously in “In Progress” and “Resolved”, irrespective of whether the lock-release event or the database change-stream event is delivered to the client first.

2.3 Implementation Requirements

- I1** The backend is implemented in Node.js, using Express for the REST API and Socket.IO for the WebSocket transport. This follows from the design need for a single event-driven runtime capable of hosting both request/response handling and long-lived WebSocket connections in the same process, which simplifies bridging MongoDB Change Stream events onto Socket.IO broadcasts.
- I2** MongoDB is deployed as a genuine multi-node Replica Set, not a single-member replica set masquerading as one. A one-member replica set would be sufficient to unlock the Change Streams API, but would not exercise the replication and automatic-failover mechanisms.
- I3** Redis is present as the shared coordination point for all cross-instance mechanisms (Socket.IO Pub/Sub adapter, distributed locks, leader election). This follows di-

rectly from the design decision to keep the API tier stateless and horizontally scalable, which in turn requires an external, shared point of coordination reachable by every replica.

- I4 The system is containerized with Docker and orchestrated with Kubernetes, validated locally on Minikube. This constraint follows from the pedagogical objective of exercising orchestration-level distributed-systems mechanisms (StatefulSets, PodDisruptionBudgets, Horizontal Pod Autoscaling, liveness/readiness separation) that would not otherwise be exercised by application code alone.
- I5 The frontend is a Vue 3 single-page application using the Composition API. This is a project-specific implementation choice rather than an externally imposed constraint: it emerged as a consequence of the design’s reliance on fine-grained reactive state derived from a continuous stream of WebSocket events.

2.4 Relevant and Non Relevant Distributed System Features

2.4.1 Transparency

Transparency is **partially relevant** to this system, and deliberately *not* pursued in full. Location and replication transparency are provided where they reduce accidental complexity: an operator’s browser never needs to know which of the (3 to 10) `api` pods it is talking to, nor which of the 3 `mongo` replicas served a given read—both are hidden behind, respectively, the Ingress with sticky sessions and the replica-set connection string. Failure transparency is provided only up to a point: a client experiencing a pod failure automatically reconnects and resynchronizes rather than surfacing a raw connection error. Concurrency transparency, however, is *intentionally not* pursued: when an incident is claimed, the system deliberately exposes this fact to other operators (via the lock indicator and disabled actions) rather than hiding it, because SOC analysts specifically need to know that a colleague is already handling a given incident to avoid duplicated effort.

2.4.2 Fault Tolerance, Dependability

The system is required to survive the loss of any one `api` pod (stateless, backed by ≥ 3 replicas) and of any one `mongo` node (backed by a 3-node Replica Set and a quorum-preserving `PodDisruptionBudget`) without interrupting service. Redis is the deliberate exception: since it holds coordination state rather than the system of record, its loss never costs an incident or a credential, but it does cost *availability* — every `api` replica fails readiness while Redis is down, so the board is unreachable until Redis returns — and, Redis having no persistence, a restart drops all active claims, which operators must take again. Data loss or silent corruption is explicitly unacceptable, which motivated optimistic locking over a “last write wins” strategy. Recovery is also fast: the leader-election lock has a 15-second time-to-live specifically so that the background escalation task resumes within seconds of a leader’s failure, rather than requiring manual restart. One honestly-documented limitation, discovered during development rather

than assumed by design, is that Redis is currently a *single, non-replicated* instance: it is a deliberate simplification appropriate to the scope of this project, and is discussed further as a known limitation in section 9.

2.4.3 Scalability

Scalability is **relevant** for the stateless API tier and **explicitly out of scope** for the data and coordination tiers. The number of concurrent operators and the request rate they generate can grow, which is why the `api` Deployment is fronted by a `HorizontalPodAutoscaler` scaling between 3 and 10 replicas on CPU utilization. MongoDB and Redis, by contrast, are scaled for *availability* (the former) or not scaled at all (the latter): the Replica Set’s 3 nodes exist to tolerate failure, not to increase read/write throughput, and no read-preference tuning is applied to route reads to secondaries. This asymmetry is a conscious trade-off given the project’s scope, not an oversight.

2.4.4 Security, Trust

Security is **deliberately minimized** in the current iteration, in favor of the project’s primary focus on distributed coordination. Operators authenticate with a username/password pair verified server-side (bcrypt-hashed credentials, stateless JWT session), but there is still a single implicit role with no differentiated permissions once authenticated. No transport encryption is configured between internal components (API, MongoDB, Redis all communicate over the cluster-internal network without TLS), and no data is encrypted at rest. These omissions are acceptable for the current scope—the system is not intended to handle sensitive data beyond incident titles and operator names—but are explicitly flagged as a direction for future work (section 9) rather than an oversight not considered during analysis.

2.4.5 Resource Sharing

Resource sharing is the **central motivation** for the entire project: multiple SOC operators must concurrently read and modify the *same* shared pool of incidents, which is precisely the scenario that requires distributed coordination rather than a simple single-user application. This single feature justifies two of the three coordination mechanisms implemented: the distributed lock, which serializes exclusive access to an individual incident (claim/release), and optimistic locking, which detects (rather than prevents) conflicting concurrent writes on the shared MongoDB collection.

2.4.6 Openness, Interoperability, Heterogeneity of Components

This feature is **not relevant** to the current scope. The system is a closed application that does not interact with external ticketing or monitoring systems, and does not expose a public, versioned API intended for third-party consumption. The architecture does not preclude such integration in the future (the REST layer could, in principle, be

consumed by an external system), but no standardization or interoperability requirement was elicited for this iteration.

2.4.7 Evolvability, Maintainability

This feature is **relevant**, and was reinforced by direct experience during development. The codebase is organized into three independently deployable units (`api/`, `web/`, `k8s/`), and the liveness/readiness probe separation (shallow vs. deep health checks) exists specifically to allow rolling updates of the API tier without unnecessary restarts or traffic loss. At the same time, maintaining hand-rolled Redis primitives (the distributed lock and the leader-election lock) proved to carry real maintenance cost: an initial, non-atomic `GET+PEXPIRE` renewal of the leadership lock was later replaced with an atomic Lua script after a split-brain risk was identified, and a manually-maintained “active users” Redis hash was later replaced with `io.fetchSockets()` after it was observed to leak stale entries across unclean pod restarts.

A third instance of the same lesson concerns the claim lock, and is the most instructive of the three because the flaw was conceptual rather than a race condition. The claim had been implemented as a pure *notification*: the only way a client could learn of it was the `incident_locked` broadcast, so the lock was absent from the state a client rebuilds when it hydrates. Two consequences followed. An operator who reloaded the page saw their own ticket back in “Open” without the `Resolve/Release` buttons the UI shows only to the lock holder, and could not re-claim it either, since a second claim attempt was answered with 409 — the legitimate holder was locked out of their own ticket until the TTL elapsed. Symmetrically, clients that hydrated at different moments disagreed about the same ticket, and nothing ever reconciled them. The remedy was to stop treating the lock as a message and start treating it as state: reading it back in the snapshot, making a re-claim by the holder idempotent, and shipping the remaining TTL so clients can expire it on their own. The general lesson — that any state a client must reconstruct has to be *readable*, not merely *announced* — is not specific to Redis, and was the least obvious of the three to notice, because nothing ever failed loudly.

These are documented here as evidence that DIY coordination primitives require deliberate care to remain correct as the system evolves, not merely to be implemented once.

2.4.8 Performance, Concurrency, Computation/Communication Efficiency, Bandwidth

This feature is **relevant** but not subject to strict quantitative requirements (e.g., no fixed latency SLA was elicited). Real-time propagation relies on MongoDB Change Streams rather than polling, and on `Socket.IO` over `WebSocket` rather than repeated HTTP requests, both chosen for communication efficiency. One trade-off surfaced during design is that every `api` pod opens its own independent Change Stream cursor against MongoDB; this keeps the cross-pod broadcast logic simple (each pod emits only to its own locally-connected clients, avoiding duplicate delivery through the Redis adapter) at

the cost of the primary node serving one concurrent change-stream cursor per pod, a cost that grows linearly as the HPA scales the API tier toward its 10-replica maximum.

2.4.9 Economy, Costs

This feature is **not a primary driver** of the design, since the project targets a local Minikube cluster for development and academic validation rather than a production deployment with a defined operating budget. Where cost-conscious choices were made, they follow from minimizing complexity rather than minimizing a monetary budget: for instance, Redis is kept as a single instance rather than a Sentinel/Cluster deployment, and MongoDB uses the minimum replica count (3) that still permits a majority-based quorum, rather than a larger cluster.

3 Design

3.1 Architecture

The system follows a three-tier, layered architecture (presentation, application, data), combined with an event-driven / publish-subscribe style for state propagation. It is deliberately *not* split into microservices: a single `api` service hosts both the REST endpoints and the WebSocket server, since the two share the same domain logic and splitting them would only add inter-service coordination without a benefit at this scale.

Two motivations drive this style, both traceable to section 2:

- **Horizontal scalability:** the `api` tier is stateless *with respect to domain state*, which lets the HPA scale it between 3 and 10 replicas. Any request can be served by any replica, since identity travels in a JWT and domain state lives only in MongoDB (incidents) and Redis (claims, leadership). Each replica still holds disposable *per-connection* state — its WebSocket sessions and its Change Stream cursor — which a client simply rebuilds by reconnecting elsewhere and rehydrating (section 3.3).
- **Near-real-time propagation:** instead of polling, clients receive events through two channels. Changes to incidents are observed by *every* replica through its own MongoDB Change Stream and pushed to that replica’s clients, so the data layer itself is the source of truth for *what changed*; changes to ephemeral state (claims, presence) start on one replica and reach the others through Redis Pub/Sub, which acts as the message broker (section 3.4).

3.2 Infrastructure

The infrastructure is composed of the following components, all containerized and orchestrated by Kubernetes (validated on Minikube):

- **Clients:** operator web browsers, running the Vue.js 3 SPA.

- **Static file server:** an Nginx container (`web` Deployment) serving the compiled Vue.js bundle; stateless, two replicas protected by a `PodDisruptionBudget`.
- **Application servers:** Express.js + Socket.IO processes (`api` Deployment), stateless with respect to domain state (section 3.1), horizontally scaled 3–10 replicas by a `HorizontalPodAutoscaler` on CPU utilization, and protected by a `PodDisruptionBudget` (`maxUnavailable: 1`, a relative bound that adapts to the current replica count).
- **Reverse proxy / load balancer:** the Kubernetes Ingress (Nginx controller, enabled as a Minikube addon), which performs path-based routing (`/` to `web`, `/api/*` and `/socket.io/*` to `api`) and cookie-based sticky sessions across the `api` replicas.
- **Database:** a 3-node MongoDB Replica Set (`StatefulSet`), the durable system of record for both incidents and operator credentials (the `incidents` and `users` collections). Each member owns a `PersistentVolumeClaim` created from the `StatefulSet`'s `volumeClaimTemplates`, so its data survives the recreation of its pod, and a `PodDisruptionBudget` (`minAvailable: 2`) prevents voluntary disruptions from taking away the majority needed to elect a primary.
- **Message broker / coordination store:** a single Redis instance, serving three distinct roles simultaneously — the Socket.IO Pub/Sub adapter (cross-pod Web-Socket broadcast), the distributed-lock store (incident claims), and the leader-election store (background task). It runs as a one-replica Deployment with no persistence and no probes of its own, which makes it the only component without redundancy: since every `api` readiness probe pings Redis, its failure takes *all* `api` replicas out of the Service at once. This single point of failure is a deliberate simplification of the project, not an oversight; replacing it with Redis Sentinel or Redis Cluster is discussed in section 9.

Network placement. In the validated deployment, all components run within a single Kubernetes cluster (a single-node Minikube instance), but each is an independently scheduled Pod communicating exclusively over the cluster's internal network. The architecture therefore does not preclude spreading these components across multiple nodes in a production-grade cluster; this was intentionally left as a deployment-time concern rather than an application-level one.

Component discovery. Services are located through Kubernetes, with no custom mechanism in the application code:

- `api`, `web` and `redis` are reached through `ClusterIP` Services and the cluster DNS (e.g., `redis.app.svc.cluster.local`). The Ingress, instead, forwards external traffic directly to the pods listed as the Service's endpoints, which is what allows its affinity cookie to pin a client to one `api` pod.

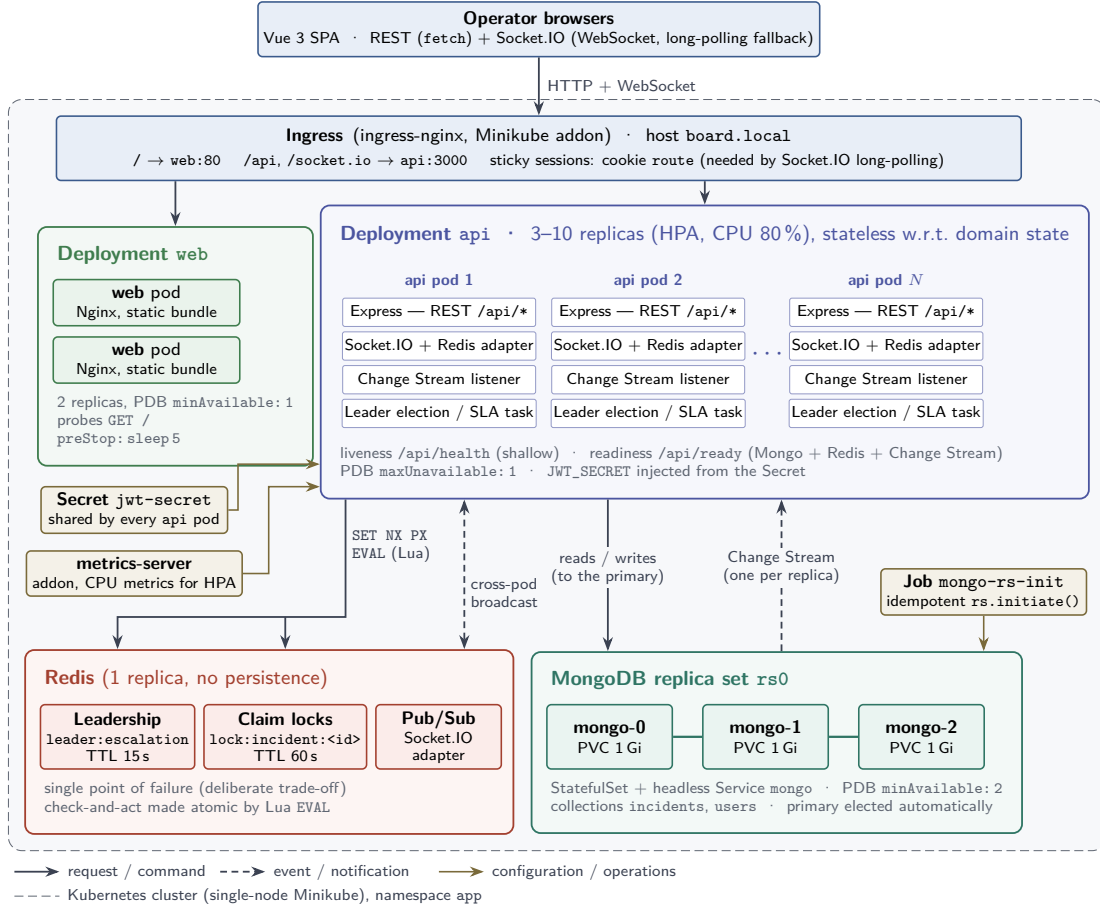


Figure 1: System architecture: request routing through the Ingress, the **web** and **api** Deployments, and the shared Redis and MongoDB backing services. Solid arrows are requests and commands; dashed arrows are events: each **api** replica receives incident changes from its own Change Stream, while claim and presence events cross replicas through Redis Pub/Sub.

- MongoDB uses a *headless* Service with the StatefulSet’s stable per-pod names (`mongo-0.mongo, ...`), because replica-set members must be reachable individually rather than load-balanced.
- Externally, the Ingress exposes a single hostname (`board.local`) with path-based routing.

Beyond this, the MongoDB driver discovers the current primary by itself from the seed list in `MONGO_URL`, and the `api` replicas never address each other: they meet only through Redis (adapter channels, `io.fetchSockets()`, the `leader:escalation` key), so the HPA can add or remove replicas without reconfiguring the others.

3.3 Modelling

Domain entities. The core entity is the **Incident** (`title`, `status` $\in \{\text{open}, \text{escalated}, \text{closed}\}$, `createdBy`, `closedBy`, `version`, `updatedAt`). Two further concepts are kept outside it. The **Operator** has persistent credentials (username and bcrypt hash), but its *presence* exists only while its WebSocket connection is open. The **Claim**, the exclusive right to work on an incident, is never written to MongoDB: it is a Redis key with a time-to-live, which is nevertheless read back into the board snapshot at every hydration (section 4.3). Figure 2 shows these entities together with the backend modules that operate on them.

Mapping to infrastructure. This distinction directly reflects each entity’s durability requirement:

- The **Incident** must survive pod restarts, rolling updates, and node failures, and is therefore stored in MongoDB, replicated across the 3-node Replica Set.
- The **Claim** and the **leadership mandate** are, by design, short-lived and self-expiring; persisting them durably would be both unnecessary and actively harmful (a durable lock that outlives its holder’s crashed connection would deadlock the resource). They are therefore stored in Redis, an in-memory store with native TTL support.
- The **Operator**’s *credentials* are persisted in MongoDB, because authentication must outlive any single connection or pod; the operator’s *presence*, by contrast, is not persisted anywhere: it exists only as `socket.data.username` on a live connection, discoverable cluster-wide on demand via `io.fetchSockets()`, and simply ceases to exist the moment the connection closes.

Domain events. `IncidentCreated`, `IncidentClaimed`, `IncidentReleased`, `IncidentResolved`, `IncidentEscalated`, and `PresenceChanged` are the events that drive the system. They are realized, respectively, as: MongoDB Change Stream notifications for the events that mutate the Incident’s persistent state (created, resolved, escalated), and direct Socket.IO broadcasts for the events that mutate only ephemeral Redis state (claimed, released, presence changed).

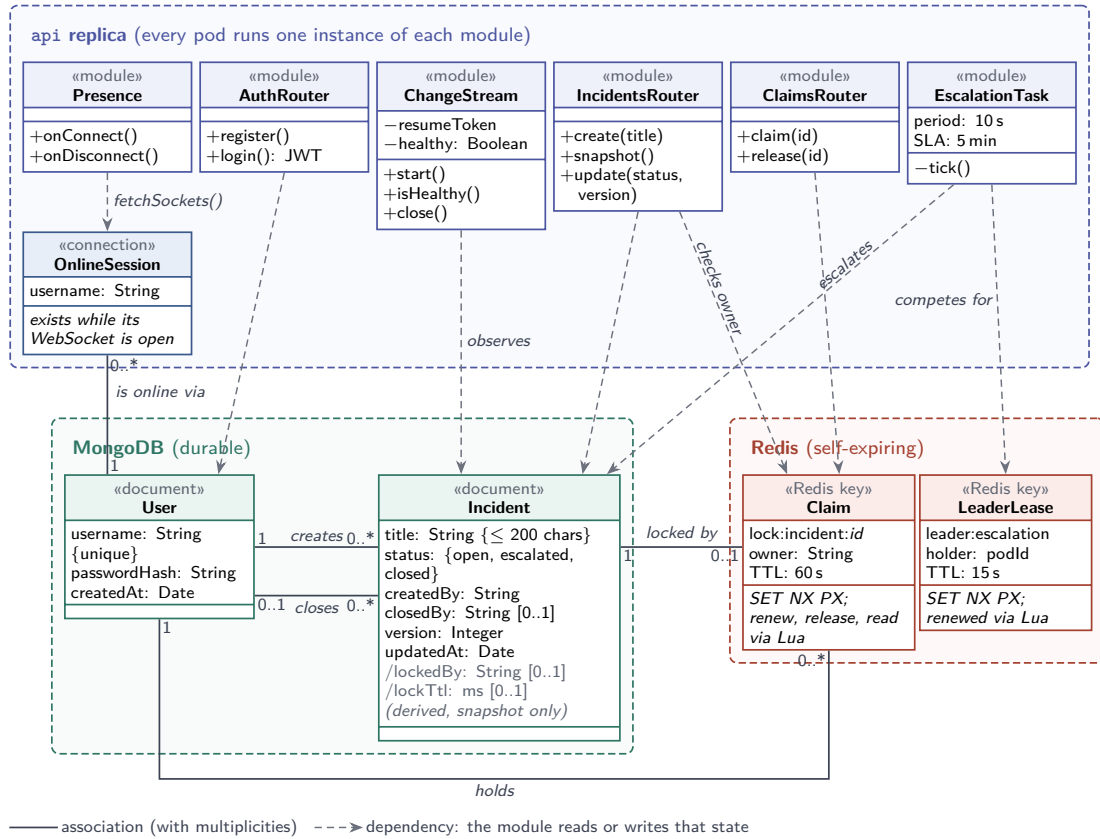


Figure 2: Class diagram.

Messages. Two distinct message categories are exchanged:

- **commands** (client → server, over REST: `POST /incidents`, `POST/DELETE .../claim`, `PATCH /incidents/:id`), which expect a definite success/failure reply and change system state.
- **events** (server → client, over Socket.IO: `incident_update`, `incident_locked/incident_unlocked`, `users_update`, `resync_required`), which are fire-and-forget notifications with no expected reply. There are no *queries* beyond `GET /api/incidents`, used to hydrate a client's local state at connection time, after every reconnect, and whenever the server itself asks for a resynchronization via `resync_required` (section 3.7) — after hydration, the client never polls again, relying entirely on the event stream to stay current.

System state. The authoritative state of the system is split across two stores by design: MongoDB holds the durable Incident collection; Redis holds transient coordination state (locks, leadership). Each connected client additionally holds a *derived*, client-local copy of a filtered view of that state (the `incidents` and `onlineUsers` reactive references in the Vue application), which is not authoritative and is kept eventually consistent with the server through the event stream and the reconnect-time resynchronization.

3.4 Interaction

Two interaction patterns coexist, cleanly separated by direction:

- **Client → server: request/response.** Every mutation (create, claim, release, resolve) is a synchronous HTTP call that yields a definite outcome (`2xx` success or `4xx/5xx` failure) before the client considers the action complete.
- **Server → client: publish/subscribe.** All state changes are broadcast to every interested client without any client explicitly requesting them, and without the server tracking who has or hasn't yet seen a given update. No server-initiated request ever targets one specific client.

Within the publish/subscribe direction, two further sub-patterns are used depending on the origin of the change, both of which were the subject of extensive tuning during development:

- Changes to **persistent** state (create, resolve, escalate) are captured by MongoDB Change Streams and broadcast via `io.local.emit()` because every `api` pod independently watches the same Change Stream and would otherwise re-broadcast an event that every other pod has *already* independently received and forwarded to its own clients, causing duplicate delivery.
- Changes to **ephemeral** state not reflected in MongoDB (claim, release, presence) are broadcast via `io.emit()`, which *does* need to cross pod boundaries through the Redis adapter, since only one pod knows the change happened: the one that

served the claim or release request, or the one where an operator’s WebSocket connected or disconnected. Resolving an incident uses both channels at once: the status change reaches clients through the Change Stream, while the release of its lock is announced with `io.emit()`.

A third case has no originating pod at all: when a claim lock simply *expires* in Redis, no request was served and no pod is aware that anything happened, so no event can be emitted. This category of change is therefore not propagated by a broadcast but *predicted* by each client independently, from the remaining time-to-live the server sends alongside every lock (section 4.6.8). Overlooking this third case was the root of a defect described in section 2.4, *Evolvability, Maintainability*.

Sequence diagrams. The four most significant interactions are shown below, each with two `api` pods so that it is visible which messages cross pod boundaries. Figure 3 covers login, the WebSocket handshake, presence and the hydration of the board. Figure 4 shows a claim with its three possible outcomes. Figure 5 shows a resolution, where the two channels run in parallel: the release of the lock travels through Redis Pub/Sub, the status change through the Change Stream of every pod. Figure 6 shows the SLA escalation run by the leader, which writes to MongoDB and leaves the notification to the Change Streams.

3.5 Behaviour

- **api pods** are stateless with respect to domain data: nothing about an incident, a claim, or a user is cached in a pod’s own memory across requests. Each pod reacts to three independent stimuli: (1) incoming REST requests, handled synchronously; (2) events pushed by its own MongoDB Change Stream subscription, handled asynchronously and forwarded only to its own locally-connected sockets; (3) an internal timer (every 10 seconds) that drives both the leader-election attempt and, if leadership is held, the escalation scan — this third stimulus is self-generated, not triggered by any external message.
- **MongoDB** accepts writes only on the elected primary, which replicates them to the two secondaries via the oplog; if the primary becomes unreachable, the remaining nodes run an automatic election (majority-based) to promote a new primary, with no application-level intervention required.
- **Redis** is purely reactive and carries no domain logic of its own: it only executes the commands (`SET NX PX`, `EVAL` of the Lua scripts, `GET`, `DEL`) issued by the `api` pods, and expires keys autonomously once their TTL elapses.
- **The Vue.js client** does not update its local state optimistically on user action; it waits for the server’s response (for the direct outcome of its own command) and for the corresponding broadcast event (for the canonical, server-confirmed state), which keeps the client’s view a strict function of what the server has actually

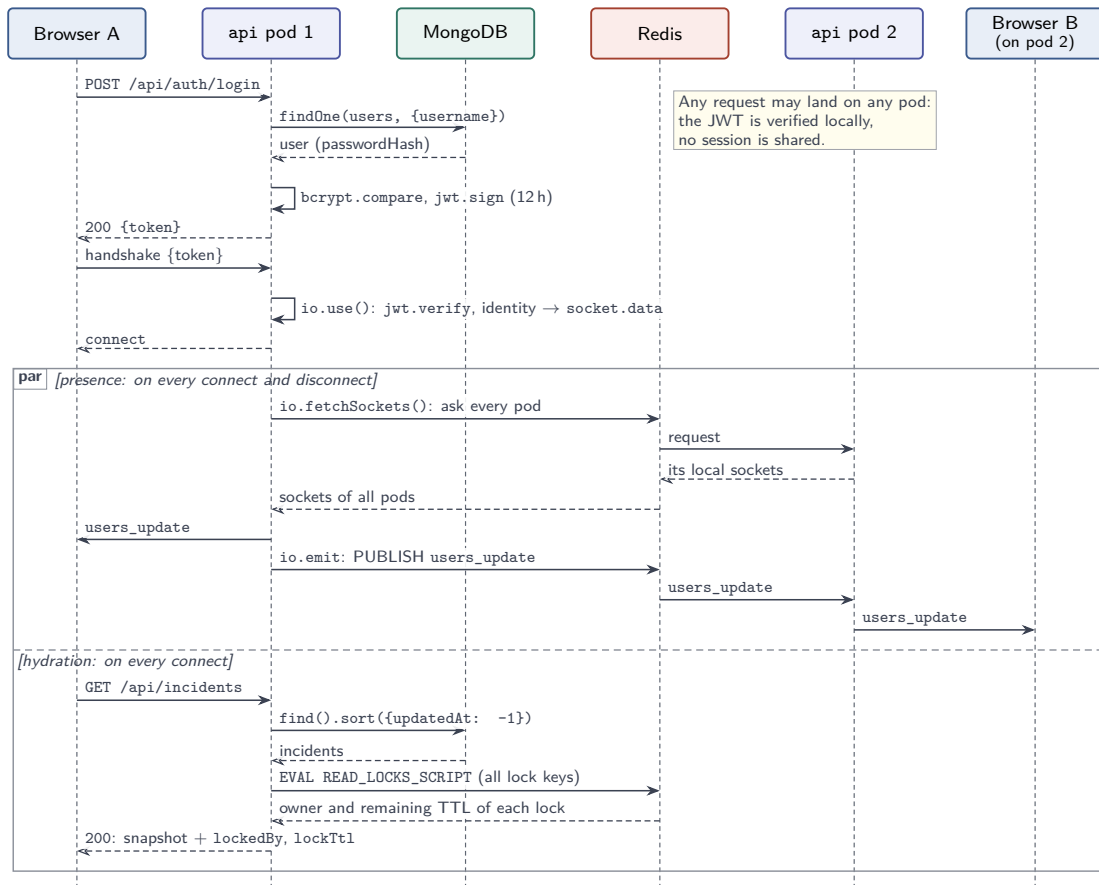


Figure 3: Login, WebSocket handshake, presence and hydration. Presence and hydration both start at every connection and run concurrently.

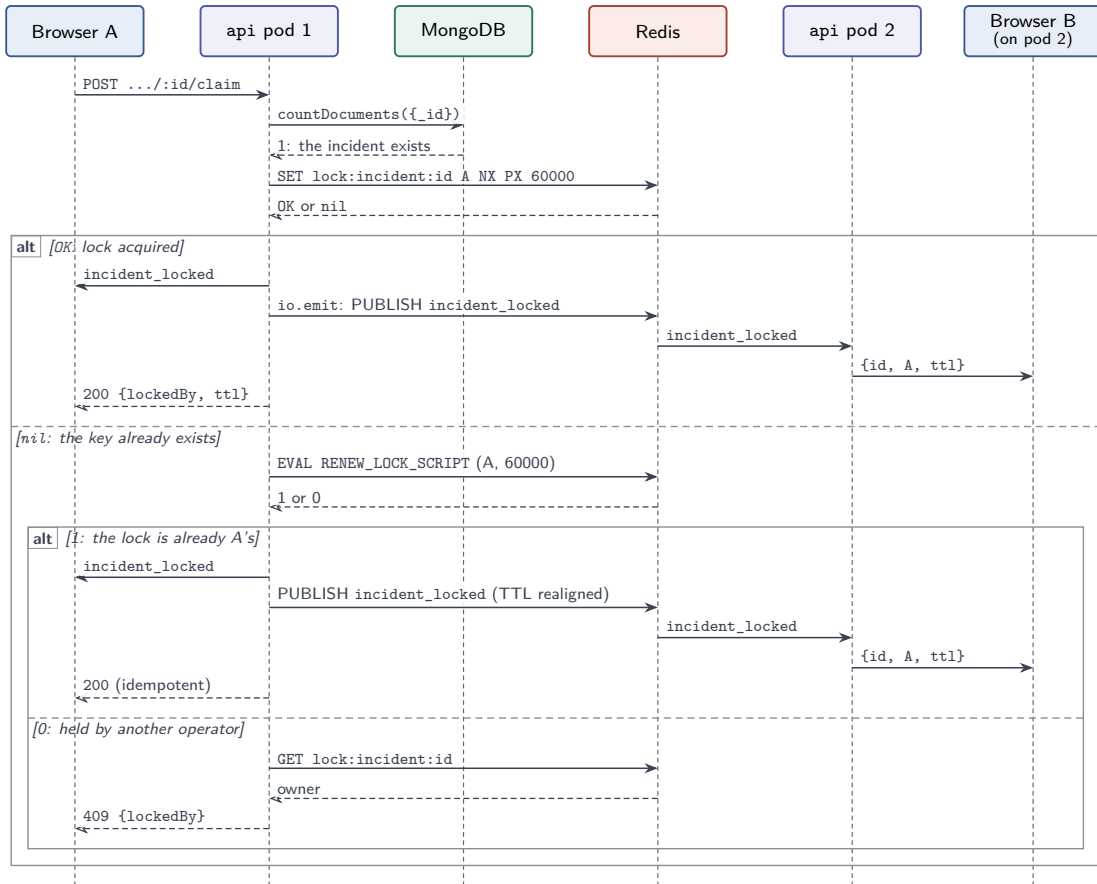


Figure 4: Claiming an incident. The claim is idempotent for the operator who already holds the lock; the 409 is reserved for a conflict with another operator.

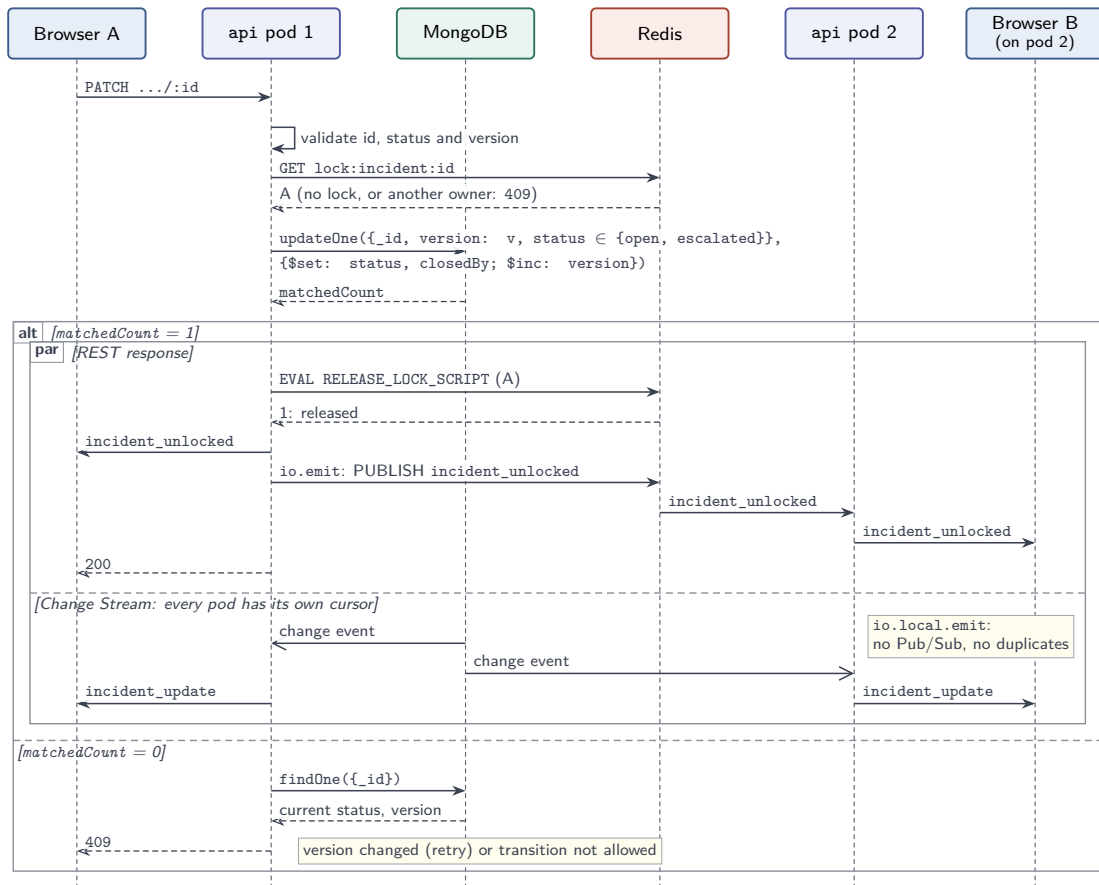


Figure 5: Resolving an incident. The PATCH requires owning the lock, and the update is conditioned on both the version and the allowed source status.

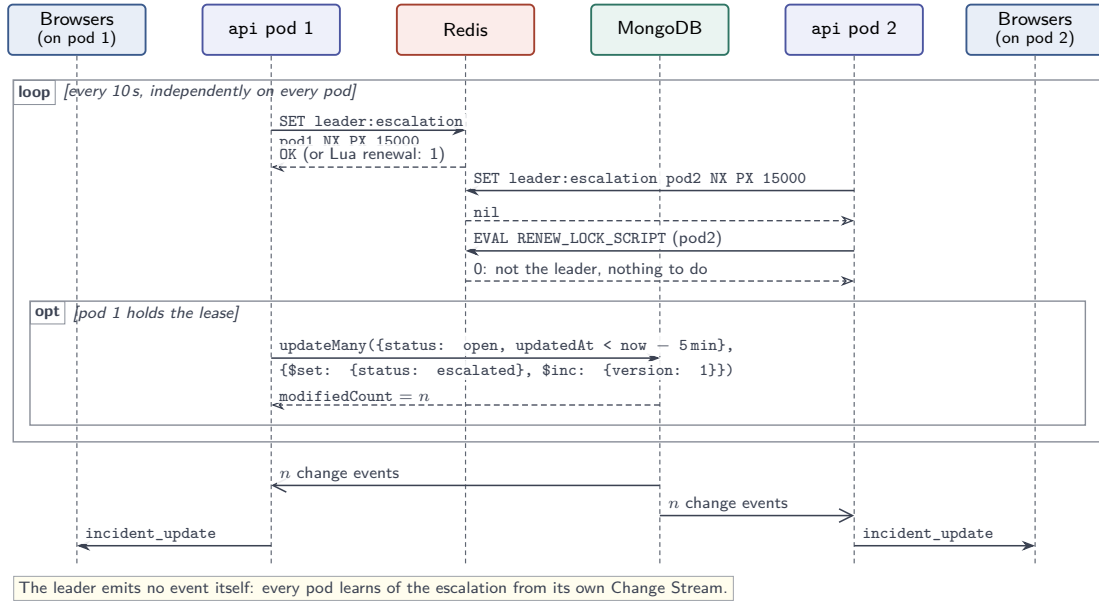


Figure 6: SLA escalation. Every pod competes for the lease at every tick; only the leader writes, and every pod notifies its own clients from its Change Stream.

confirmed, rather than a prediction that could be wrong and require a visible correction later.

State diagrams. Four components have an explicit lifecycle worth modelling. An **incident** (fig. 7) combines its durable status in MongoDB with the ephemeral claim in Redis: the two dimensions are independent, which is why an incident can be escalated while someone is working on it. Each **api pod** has two internal state machines: its **Change Stream** (fig. 8), which decides whether the pod is ready to receive traffic, and its role in the **leader election** (fig. 9). Finally, each **client session** (fig. 10) moves between connecting, hydrating from the REST snapshot and following live events.

3.6 Data and Consistency Issues

What is stored, and where. Incidents are the only data that must persist beyond a single request or connection; they are stored in **MongoDB** as documents. The document model was chosen because incidents are naturally semi-structured records whose schema evolved incrementally over the project’s development (e.g., the **escalated** status and the **closedBy** field were both introduced after the initial design) without requiring a formal migration step. Coordination data (locks, leadership) is stored in **Redis** as plain key-value pairs, chosen because the access pattern is a pure read/write/expire on independent keys with no relational structure, and because lock contention needs sub-millisecond latency that an in-memory store provides naturally.

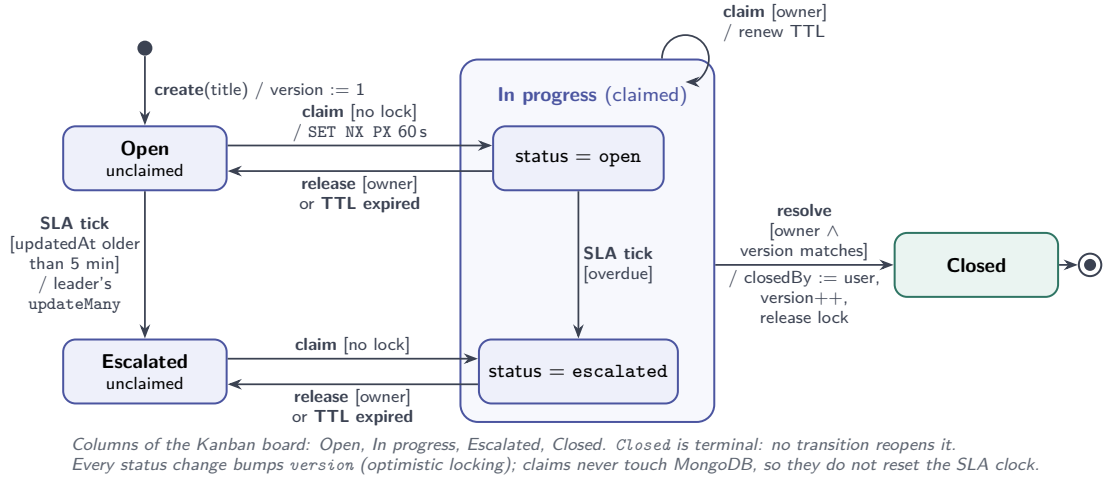
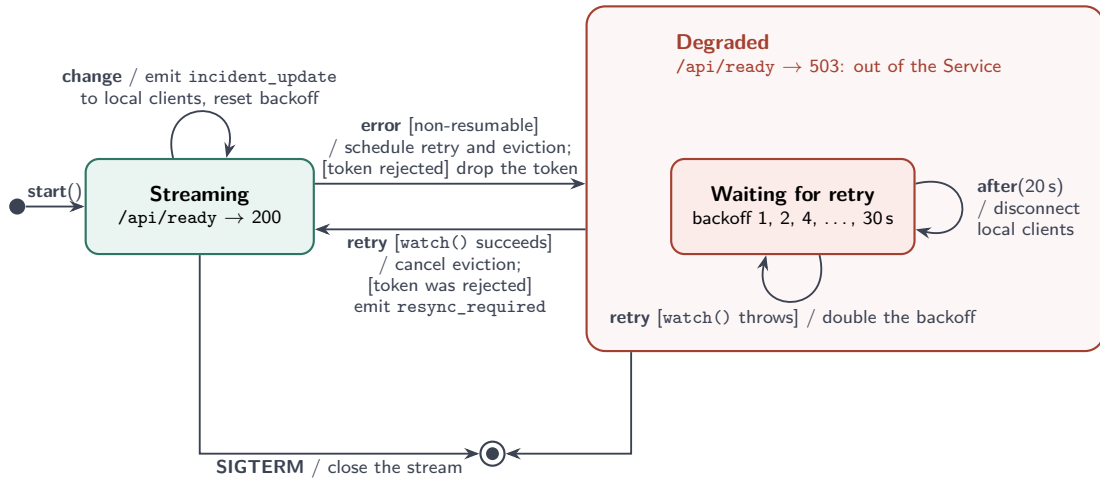
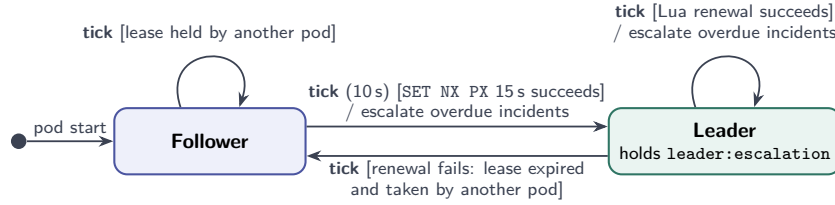


Figure 7: Incident lifecycle. The *In progress* composite state corresponds to the claimed incidents; its two substates keep the status that the incident returns to when the claim is released or expires.



Liveness stays shallow in both states: Kubernetes takes a degraded pod out of the Service but never restarts it. Evicted clients reconnect through the Service, which no longer lists this pod, and rehydrate.

Figure 8: Change Stream of a single *api* pod. Only the *Streaming* state passes the readiness probe; the retry reschedules itself even when `watch()` fails synchronously.



A crashed or terminated leader does not release the lease: another pod takes over within 15 s plus one tick. Without a fencing token, a leader paused for more than 15 s may overlap with its successor; the escalation is idempotent.

Figure 9: Leader election, as seen by a single `api` pod at every 10-second tick.

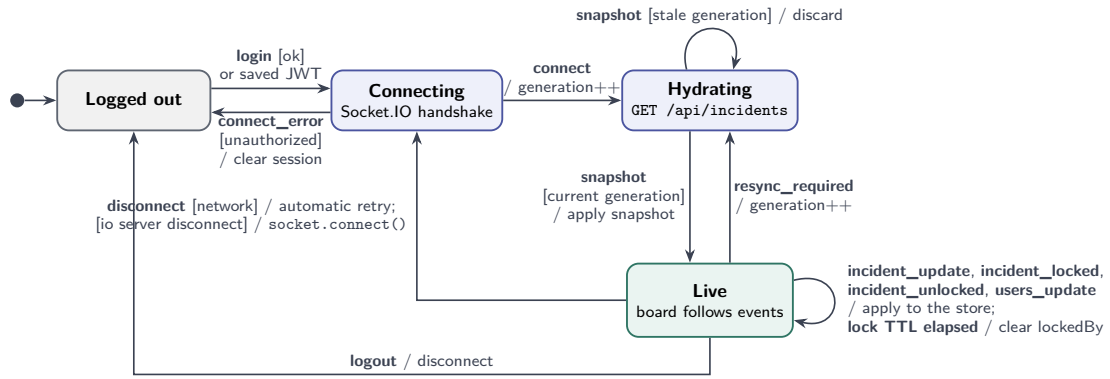


Figure 10: Client session in the browser. The *generation* counter discards snapshots that arrive after a newer resynchronization has started.

Query and write patterns. `api` pods are the only components that ever query or write MongoDB and Redis; the client never talks to either data store directly, always mediating through the REST/Socket.IO layer. Concurrent *reads* are unrestricted and require no coordination (both MongoDB and Redis handle concurrent reads natively). Concurrent *writes* are expected and must be explicitly coordinated: this is precisely the motivation behind the two concurrency-control mechanisms of section 2.2 — optimistic locking detects conflicting concurrent writes to the *same* Incident document, and the distributed lock prevents two operators from concurrently claiming the same Incident in the first place.

Shared data. The Incidents collection is, by design, shared read/write state across every `api` replica — this sharing is the entire point of keeping the application tier stateless and horizontally scalable. The same is true of the Redis keyspace: coordination only works if every replica observes the same lock and leadership state, which is why a single shared Redis instance (rather than one per pod) is a hard requirement, not an implementation convenience.

Consistency model. The system provides **strong consistency** for writes to the primary (a successful write is immediately visible to any subsequent read against the primary) and **eventual consistency** for propagation to connected clients: a change is always durably committed to MongoDB *before* any client is notified of it, but different clients may observe the notification at slightly different times depending on Change-Stream and network latency. This asymmetry is a deliberate, accepted trade-off: the domain (a SOC dashboard) tolerates sub-second staleness in what operators *see*, but never tolerates staleness or loss in what is actually *recorded* as the incident’s state.

3.7 Fault-Tolerance

Replication. MongoDB is replicated across a 3-node Replica Set specifically to survive the loss of one node while preserving both durability and write availability. Redis, in contrast, is *not* replicated: it runs as a single instance, a deliberate and explicitly documented simplification (section 2.4, *Fault Tolerance, Dependability*) rather than an oversight, justified by Redis holding only coordination state rather than the system’s durable record. Not all of that state is re-derivable, though: the leadership is, since after a Redis restart the key is simply absent and a new leader is elected at the next 10-second tick of the election loop, but the active claims are not — nothing reconstructs them, and the operators who held them must claim their tickets again. The cost of the simplification is therefore an availability outage for as long as Redis is down plus the loss of in-progress claims, never a loss of durable data. The `api` tier is “replicated” in a different sense: its 3–10 pods are interchangeable stateless workers, not holders of partitioned or replicated data.

Heartbeats, timeouts, and retries. Several independent mechanisms play this role at different layers:

- Kubernetes **liveness** (`/api/health`) and **readiness** (`/api/ready`) probes act as an external heartbeat on each `api` pod and its dependencies, respectively — deliberately kept shallow and deep, so a dependency outage only removes a pod from traffic (readiness) rather than restarting a healthy process (liveness). The readiness probe reports not only on reachability of MongoDB and Redis but also on the health of the pod’s own Change Stream, precisely because a pod whose stream has died is the worst kind of casualty: perfectly healthy from Express’s point of view, yet silently serving clients that will never receive another update.
- Redis-backed locks (claim, leadership) use a **TTL as an implicit timeout**: rather than requiring an explicit failure detector to notice a crashed lock holder, the lock simply expires and becomes acquirable again — a lease-based approach that trades a bounded worst-case delay (up to the TTL) for the absence of any active monitoring logic.
- Socket.IO’s own `engine.io` ping/pong protocol is a heartbeat between client and server that detects dead connections independently of TCP-level timeouts, which is what makes presence tracking via `io.fetchSockets()` self-correcting.
- The Socket.IO client’s automatic reconnection is a **retry** mechanism, explicitly paired with the resynchronization-generation counter (NFR7) so that retries cannot corrupt the client’s perceived state ordering.
- The Change Stream is itself **retried** server-side. Its **error** handler marks the pod not-ready, closes the broken cursor and rebuilds it after an exponentially increasing delay (1s doubling up to a 30s ceiling), reopening it with `resumeAfter` set to the last processed resume token so that changes occurring during the outage are replayed rather than lost. The backoff counter is reset only when an event is actually *received*, not when the cursor is created: `watch()` always succeeds, so resetting on creation would degenerate into a tight retry loop against a persistently failing stream. If the resume token is itself rejected — typically because the interruption outlasted the oplog window (error codes 286/280) — the stream restarts from the present and the pod emits `resync_required` to its locally-connected clients, converting an unrecoverable gap in the event feed into a full re-hydration instead of silent divergence.

Error handling. Every REST handler wraps its logic in a `try/catch` block and translates failures into a specific HTTP status (409 for a lock or version conflict, 503 for a not-yet-ready dependency, 500 for an unexpected error) rather than letting the process crash, with one exception: the incident-creation handler (`POST /api/incidents`) omits the `try/catch` wrapper present in every other route, an inconsistency not caught during development. Failure to reach MongoDB or Redis *at startup* is treated as fatal (`process.exit(1)`), by design: Kubernetes then restarts the pod into a clean, fully-initialized state instead of leaving it serving traffic from a half-initialized process. On the frontend, most mutating actions surface a visible notification on failure — including

when the server’s response body is not valid JSON — a defensive behaviour added after a real failure mode was observed during development, where a stale deployed image caused a route to 404 and the resulting parse error was silently swallowed, leaving the operator with no feedback at all. This defensive pattern (`try/catch` plus a `response.ok` check) is applied consistently to `claimIncident`, `releaseClaim`, and `resolveIncident`, but was not carried over to `createIncident`, which neither checks the response status nor catches a network failure (section 5.2, FR10).

3.8 Availability

Caching. No read cache is used for Incident data: every `GET` reads MongoDB directly. This is a deliberate simplicity choice — the read volume expected from a SOC dashboard with a handful of concurrent operators does not justify the added complexity of cache invalidation. Redis, despite superficially resembling a cache, is never used to cache MongoDB reads in this system; its only roles are coordination (locks, leadership) and message brokering (the `Socket.IO` adapter).

Load balancing. The Ingress load-balances HTTP and WebSocket traffic across the ≥ 3 `api` replicas, using cookie-based **sticky sessions** specifically because `Socket.IO`’s HTTP long-polling fallback transport requires successive requests within one handshake to reach the *same* backend process; a plain stateless REST call would not need this affinity, but the shared Ingress rule applies it uniformly for simplicity. This mechanism works together with the `HorizontalPodAutoscaler`, which keeps the pool of backends elastic under load (section 2.2).

Behaviour under network partitioning. Several partition scenarios are explicitly handled, each favouring a different side of the CAP trade-off depending on what is partitioned:

- If an `api` pod loses connectivity to Redis or MongoDB, or its Change Stream dies without being immediately re-establishable, `/api/ready` starts failing and Kubernetes removes the pod from the Service’s endpoint list — it stops receiving traffic but is *not* killed, and rejoins automatically once connectivity is restored. This favours **consistency and correctness** (no pod serves requests it cannot fulfil correctly) over that individual pod’s availability.
- If a MongoDB secondary is partitioned from the primary, it simply falls behind and resynchronizes once the partition heals, without disrupting the primary’s availability.
- If the primary itself is partitioned from the majority of the Replica Set, it steps down to avoid a split-brain scenario (two nodes both believing they are primary), again favouring **consistency (CP)** over the availability of the minority side — the design explicitly accepts that the minority partition becomes unavailable for writes rather than risk divergent, unreconcilable incident data.

3.9 Security

Authentication. A single-factor username/password mechanism is implemented: an operator registers a username and password, the password is hashed with bcrypt before being persisted, and login issues a stateless JSON Web Token (JWT) signed with a secret shared across all `api` replicas via a Kubernetes `Secret`. The token is required on every subsequent REST call (**Authorization:** `Bearer` header) and on the Socket.IO handshake, and is verified independently by whichever pod happens to receive the request — no server-side session state is kept, which preserves the stateless, horizontally-scalable nature of the `api` tier. This remains a scoped, deliberate simplification (already discussed in section 2.4, *Security, Trust*) rather than an oversight: it is single-factor authentication (username/password are both “something you know”), not multi-factor, and there is no password-reset flow, account lockout, or rate limiting on login attempts — the project’s focus is on distributed coordination, and the domain data involved (incident titles, operator names) does not warrant the added complexity of a full identity provider for the current iteration.

Authorization. There is a single implicit role (“operator”) with no differentiated permissions — every authenticated client can create, claim, release, and resolve any incident. One narrow exception exists: possession of an incident’s Redis lock *does* function as a form of **capability-based authorization** for a single specific action (resolving or releasing that one incident): the server rejects a resolution attempt from anyone other than the current lock holder (section 2.2, NFR2). Unlike in an earlier iteration, the lock’s stored value is no longer an arbitrary, client-declared string: it is the username extracted server-side from the caller’s verified JWT, so the capability (the lock) is now anchored to an authenticated identity rather than to an unverified claim.

Cryptography. Cryptographic primitives are now used for authentication — bcrypt for password hashing and HMAC-signed JWTs for session tokens — but nowhere else in the system: communication between the `api` tier and MongoDB/Redis remains unencrypted (cluster-internal traffic only), and no data is encrypted at rest. Transport-level and at-rest encryption remain explicitly scoped-out concerns rather than unconsidered gaps, and are listed as a candidate direction in section 9.

4 Implementation

This section documents the technology-dependent choices made while translating the design of section 3 into a running system. Each subsection maps directly to one of the implementation dimensions identified in the project template and is grounded in the actual source code of the three project directories (`api/`, `web/`, `k8s/`).

4.1 Network Protocols

Two application-level protocols coexist, each chosen for a specific interaction pattern:

HTTP/1.1 over TCP is used for all **command** (mutation) interactions from the client to the server. The REST API exposed by Express.js defines five endpoints:

- **POST /api/incidents** — create a new incident;
- **GET /api/incidents** — retrieve the full incident list, each document enriched with the current holder of its claim and that claim’s remaining time-to-live (used at connect time, on every reconnect, and on server-requested resynchronization);
- **POST /api/incidents/:id/claim** — acquire the distributed lock on an incident;
- **DELETE /api/incidents/:id/claim** — explicitly release a held lock;
- **PATCH /api/incidents/:id** — resolve an incident (with optimistic-locking version check and server-side lock verification).

HTTP was chosen because every mutation requires an unambiguous, synchronous success-or-failure response (e.g., “did I get the lock or not?”), which request/response semantics provide naturally. Two additional endpoints serve as Kubernetes health probes: **GET /api/health** (liveness, shallow) and **GET /api/ready** (readiness, deep — pings both MongoDB and Redis, and additionally reports 503 if the pod’s own Change Stream is not currently active).

WebSocket (via Socket.IO) is used for all **event** (notification) interactions from the server to connected clients. The Socket.IO library negotiates a WebSocket upgrade after an initial HTTP handshake, falling back to HTTP long-polling if WebSocket is unavailable. The server emits five event types:

- **incident_update** — carries a MongoDB Change Stream document (insert or update) describing a persistent state change;
- **incident_locked / incident_unlocked** — notifies all clients of ephemeral lock state changes; **incident_locked** also carries the claim’s time-to-live, so that each client can expire the lock locally when no event will announce its natural expiry;
- **users_update** — broadcasts the current list of connected operators;
- **resync_required** — asks the pod’s own clients to re-hydrate from **GET /api/incidents**, emitted when the pod knows it has lost a segment of the change feed and cannot replay it (section 3.7).

WebSocket was chosen over polling because the design requires near-real-time propagation of every state change to every connected client; a polling strategy would either waste bandwidth (short intervals, many no-op responses) or introduce unacceptable latency (long intervals, stale dashboards).

Redis protocol (RESP). The `api` pods communicate with Redis using the native Redis Serialization Protocol over TCP, mediated by the `redis` npm package (v6). This includes standard commands (`SET`, `GET`, `DEL`, `PING`) and Lua script evaluation (`EVAL`) for atomic lock operations.

MongoDB Wire Protocol. The `api` pods communicate with the MongoDB Replica Set using the MongoDB binary wire protocol over TCP, mediated by the official `mongodb` npm driver (v7). The connection string enumerates all three replica-set members (`mongo-0.mongo:27017`, `mongo-1.mongo:27017`, `mongo-2.mongo:27017`) with `replicaSet=rs0`, enabling automatic primary discovery and failover-transparent reconnection.

4.2 In-Transit Data Representation

JSON is used as the sole serialization format for all inter-component communication that crosses a network boundary in the application layer:

- **REST API:** both request bodies (e.g., `{"title": "...", "status": "open", "createdBy": "alice"}`) and response bodies are JSON. The Express.js middleware `express.json()` parses incoming bodies, and every route handler returns a JSON object via `res.json()`.
- **Socket.IO events:** each emitted event carries a JSON payload. For `incident_update`, the payload is the raw MongoDB Change Stream document (which is already a JSON-like BSON structure, automatically serialized to JSON by the driver). For `incident_locked` the payload is `{id, lockedBy, ttl}` and for `incident_unlocked` simply `{id}`; for presence, a plain array of username strings; `resync_required` carries no payload at all, being a pure signal. The `ttl` field is deliberately a *relative* duration in milliseconds rather than an absolute expiry timestamp: an absolute deadline would only be interpretable by a client whose clock agrees with the cluster's, and a few seconds of skew would make locks expire visibly too early or too late.
- **MongoDB ↔ driver:** the driver uses BSON (Binary JSON) on the wire, which is transparently serialized/deserialized by the `mongodb` npm package; the application code only ever manipulates plain JavaScript objects.
- **Redis ↔ driver:** Redis values are plain UTF-8 strings (usernames, pod identifiers); no structured serialization is needed because each key holds a single scalar value.

JSON was chosen for simplicity, native support in both JavaScript runtimes (Node.js and the browser), and human readability during debugging. The overhead of JSON over a binary format like Protocol Buffers is negligible at the expected message volumes of a SOC dashboard.

4.3 Database Querying

The system uses **MongoDB**, a document-oriented NoSQL database, queried through the official Node.js driver's fluent API rather than through a raw query language.

Write operations. Three write patterns are used:

- `insertOne({...})` for incident creation, which atomically assigns a `version: 1` and an `updatedAt` timestamp;
- `updateOne({_id, version}, {$set: ..., $inc: {version: 1}})` for single-incident resolution, implementing the optimistic-locking pattern: the `version` field in the query filter ensures the update only applies if no concurrent modification has incremented it since the client last read the document;
- `updateMany({status: 'open', updatedAt: {$lt: threshold}}, ...)` for the background escalation task, which bulk-transitions all stale open incidents to `escalated` in a single round-trip.

Read operations. A single MongoDB read pattern is used: `find({}).sort({updatedAt: -1}).toArray()`, which retrieves the entire incidents collection sorted by last-update time. Because a claim lives in Redis and not in the document, this result alone would describe only *half* of each incident's observable state, so the handler then reads the corresponding lock keys from Redis and merges the holder and the remaining TTL into each returned document. The merge is what makes the REST snapshot and the subsequent event stream describe the same state for every client, no matter when it connects; the two are read in that order (documents first, locks second) so that a lock acquired in between is reported as held rather than omitted. The enriched snapshot is fetched on connection, on every reconnection and on `resync.required`; after hydration, the client relies exclusively on the event feed.

Change Streams. The `watch([], {fullDocument: 'updateLookup'})` call opens a tailable cursor on the collection's oplog, streaming every insert and update as a structured change event that includes the full post-image of the document. This eliminates the need for any polling logic and provides the reactive bridge between the data layer and the real-time notification layer. Since this cursor is the *only* channel through which a client learns of anything after hydration, its failure is treated as a first-class fault rather than left to chance: the `_id` of each delivered event is retained as a resume token and passed as `resumeAfter` when the cursor is rebuilt after an error, and the pod declares itself not-ready for as long as no cursor is open (section 3.7). The MongoDB driver already retries *resumable* errors internally (a primary failover, for instance), so the application-level handler only ever runs for the non-resumable remainder.

Redis querying. Redis is queried with single-key commands (`SET NX PX`, `GET`, `DEL`) and three Lua scripts executed via `EVAL`:

- **RENEW_LOCK_SCRIPT**: atomically checks that the value of a key matches an expected owner and, only if so, extends its TTL — preventing the split-brain window that a non-atomic **GET+PEXPIRE** sequence would introduce. It is used both to renew the leadership mandate (owner = the pod identifier) and to renew a claim on behalf of the operator who already holds it;
- **RELEASE_LOCK_SCRIPT**: atomically checks that the value of a claim lock matches the releasing operator and, only if so, deletes the key — preventing an operator from accidentally deleting a lock that has since been acquired by someone else after the original lock expired;
- **READ_LOCKS_SCRIPT**: the only multi-key script, used to hydrate the board snapshot. It iterates over the lock keys of all incidents and returns, for each key that actually exists, the triple (key, owner, remaining TTL). Reading owner and TTL inside a single **EVAL** is what guarantees they refer to the same instant: an equivalent **GET+PTTL** pair issued from the application could observe an owner that has already expired by the time its TTL is read. It also collapses what would otherwise be one round-trip per incident into one, so the cost of hydration does not grow with the size of the board.

All three scripts exploit Redis’s single-threaded execution model: no other command can interleave between the **GET** check and the **PEXPIRE/DEL/PTTL** action within the same **EVAL** invocation.

4.4 Authentication

A single-factor, stateless authentication mechanism is implemented. An operator first registers via **POST /api/auth/register** with a username and a password (minimum 8 characters); the password is hashed with **bcrypt** (10 salt rounds) before being stored in the **users** MongoDB collection, which carries a unique index on **username** to prevent duplicate registrations even under concurrent requests from different **api** pods. **POST /api/auth/login** verifies the submitted password against the stored hash and, on success, issues a JWT (**jsonwebtoken**, 12-hour expiry) whose payload identifies the username; both endpoints return the same generic error message for a non-existent user and for a wrong password, to avoid leaking which usernames are registered.

The JWT is signed with a secret (**JWT_SECRET**) injected into every **api** pod from the same Kubernetes **Secret** (**jwt-secret.yaml**). This is what keeps authentication compatible with the system’s horizontally-scaled, stateless design: no session state is held by any individual pod, so a token issued by one replica can be verified by any other replica that later receives a request from the same client, with no sticky session required. On the client side, the token is attached as an **Authorization: Bearer** header on every REST call and as **socket.handshake.auth.token** on the Socket.IO handshake; an Express middleware (**authenticate**) and a Socket.IO middleware (**io.use()**) both verify it independently and reject the request/handshake if it is missing, malformed, or expired.

Identity derived from the verified token, not from client-supplied data, is used everywhere an operator’s identity previously had to be trusted: `createdBy` on incident creation, the lock owner on claim/release, and `closedBy` on resolution are all now read from `req.user.username` (REST) or `socket.data.username` (Socket.IO) rather than from the request body or handshake payload — closing an impersonation gap where a client could previously act, claim, or resolve incidents under any username of its choosing simply by asserting it. This remains a deliberately scoped simplification: it is single-factor authentication (username and password are both “something you know”, not a second independent factor), and there is no password-reset flow, account lockout, or brute-force rate limiting on `/api/auth/login` for the current iteration.

4.5 Authorization

No role-based or attribute-based access control is implemented. All authenticated operators share the same implicit role and can perform every available action (create, claim, release, resolve). There is, however, one narrow form of **capability-based authorization** enforced server-side: possession of an incident’s Redis lock functions as the sole authorization token for resolving or releasing that specific incident.

Concretely, the `PATCH /api/incidents/:id` handler checks the current lock owner against the authenticated username of the caller (`req.user.username`, derived from the verified JWT) before allowing the update to proceed; if they do not match (i.e., someone other than the lock holder is attempting to resolve), the request is rejected with HTTP 409, regardless of what the client UI displays. Similarly, the `DELETE /api/incidents/:id/claim` handler uses the `RELEASE_LOCK_SCRIPT` to ensure only the lock holder can release it. This is still authorization by *possession of a token* (the lock’s stored value) rather than a distinct RBAC/ABAC layer, but that stored value is, since the authentication work described in section 4.4, itself an authenticated identity rather than an arbitrary client-declared string.

4.6 Technological Details

4.6.1 Backend Runtime and Frameworks

- **Node.js 20** (Alpine-based Docker image, `node:20-alpine`): chosen as the application runtime because its single-threaded, event-loop-based execution model is well suited to hosting both synchronous request handlers and long-lived WebSocket connections in the same process, without the thread-management overhead of a multi-threaded server.
- **Express.js 5**: provides the HTTP routing layer. An `express.Router()` instance mounts all endpoints under the `/api` prefix; the `express.json()` middleware handles request body parsing; and `cors()` is configured with the `CORS_ORIGIN` environment variable to restrict cross-origin requests in deployment.
- **Socket.IO 4**: provides the WebSocket transport layer, with automatic fallback to HTTP long-polling if WebSocket upgrade fails. The `@socket.io/redis-adapter`

(v8) bridges all Socket.IO instances across the `api` pods so that events emitted with `io.emit()` are propagated cluster-wide via Redis Pub/Sub.

- **mongodb 7** (official Node.js driver): provides the connection to the MongoDB Replica Set, including Change Streams support and automatic primary failover handling.
- **redis 6** (Node.js client): provides the connection to Redis for distributed locks, leader election, and the Socket.IO adapter's Pub/Sub channels. Two client instances are created (`pubClient` and `subClient`), as required by the Redis adapter which needs dedicated publish and subscribe connections.

4.6.2 Frontend Framework and Libraries

- **Vue.js 3** with the Composition API (`<script setup>`): the SPA framework. The entire dashboard is a single `App.vue` component using Vue's reactivity system (`ref`, `computed`) to derive filtered views of the incident list and KPI counters from a single reactive `incidents` array that is kept current by the Socket.IO event stream.
- **Vite 8**: the build tool and development server. The production build (triggered by `npm run build` inside the multi-stage Dockerfile) produces a static bundle served by Nginx.
- **socket.io-client 4**: the browser-side Socket.IO library. The connection is established with `io('/', {path: '/socket.io', auth: {token}})`, which leverages the Ingress's path-based routing to reach the `api` service; the token is the same JWT used for REST calls, verified by the server on the handshake (section 4.4).
- **vue-toastification 2**: provides non-blocking toast notifications for all real-time events (new tickets, escalations, resolutions by others) and for local operation failures (claim conflicts, version conflicts, network errors). Configured in `main.js` with bottom-right positioning, 5-second timeout, and a progress bar.
- **Chart.js 4** (via `vue-chartjs 5`): renders the bar chart in the "Incidents Status Overview" card, with reactive data bindings to the computed KPI properties so the chart updates automatically as the incident counts change.

4.6.3 Containerization

Both application components use **Docker** multi-stage or single-stage builds:

- **API image**: a single-stage build from `node:20-alpine`. The Dockerfile copies `package*.json` first and runs `npm ci --only=production` to leverage Docker's layer cache, then copies the application code. The entrypoint is `node index.js`.

- **Web image:** a multi-stage build. Stage 1 (`node:20-alpine`) installs dependencies and runs `npm run build` to compile the Vue SPA into static assets. Stage 2 copies the `dist/` output into an `nginx:alpine` image, producing a minimal container that serves only static files on port 80.
- **MongoDB and Redis:** use the official upstream images (`mongo:6` and `redis:7`) without custom Dockerfiles.

Both custom images set `imagePullPolicy: Never` in their Kubernetes manifests, instructing Kubernetes to use the locally-built images available in Minikube's Docker daemon rather than attempting to pull them from a remote registry.

4.6.4 Orchestration (Kubernetes)

The Kubernetes manifests in `k8s/` define the following resources, all deployed in the `app` namespace:

- **API Deployment + Service** (`api.yaml`): a `Deployment` with 3 initial replicas, each running the `api:latest` image. Environment variables configure the MongoDB connection string (enumerating all three replica-set members), the Redis URL, and the CORS origin. The liveness probe (`/api/health`, shallow) and readiness probe (`/api/ready`, deep) are deliberately separated: a dependency outage only removes the pod from the Service's endpoint list (readiness failure) without restarting a healthy process (liveness would pass).
- **HorizontalPodAutoscaler** (`hpa.yaml`): targets the `api` Deployment, scaling between 3 and 10 replicas based on an 80% average CPU utilization threshold.
- **MongoDB StatefulSet + Headless Service + ConfigMap + Job + PDB** (`mongo.yaml`): the most complex manifest, comprising five resources:
 - A **headless Service** (`clusterIP: None`) that provides stable per-pod DNS names (`mongo-0.mongo`, `mongo-1.mongo`, `mongo-2.mongo`), necessary because replica-set members must be individually addressable.
 - A **ConfigMap** containing the `rs.initiate()` JavaScript invocation with the three member hostnames.
 - A **StatefulSet** with 3 replicas, each running `mongo:6` with `--replSet rs0 --bind_ip_all`. Each pod has a `volumeClaimTemplate` requesting 1 GiB of persistent storage, ensuring data survives pod restarts. Liveness and readiness probes execute `db.adminCommand('ping')` via `mongosh`.
 - A one-shot **Job** (`mongo-rs-init`) that waits for all three members to be reachable, checks whether the replica set is already initialized, and if not, runs the `rs.initiate()` script and waits for a primary to be elected. The `backoffLimit: 5` and `restartPolicy: OnFailure` ensure the initialization is retried if it fails due to a transient readiness issue.

- A **PodDisruptionBudget** (`minAvailable: 2`) that prevents voluntary evictions (e.g., node drains, rolling updates) from disrupting more than one MongoDB pod at a time, preserving the quorum needed for automatic primary election.
- **Redis Deployment + Service** (`redis.yaml`): a single-replica **Deployment** running `redis:7`, exposed via a **ClusterIP** Service on port 6379. No persistence, no replication — a deliberate simplification documented in section 2.4.
- **Web Deployment + Service** (`web.yaml`): a single-replica **Deployment** running the Nginx-based `web:latest` image, exposed via a **ClusterIP** Service on port 80.
- **Ingress** (`ingress.yaml`): an Nginx Ingress resource configured for host `board.local` with three path-based routing rules:
 - `/` → `web:80` (static frontend assets);
 - `/api` → `api:3000` (REST endpoints);
 - `/socket.io` → `api:3000` (WebSocket transport).

Cookie-based sticky sessions are enabled via annotations (`nginx.ingress.kubernetes.io/affinity: cookie`, cookie name `route`, 48-hour expiry) specifically because Socket.IO's HTTP long-polling fallback requires successive requests within the same handshake to reach the same backend pod.

4.6.5 Concurrency Control Implementation

Two concurrency control mechanisms are implemented, each targeting a different failure mode:

- **Distributed lock (Redis SET NX PX)**: used for incident claiming. The lock key follows the pattern `lock:incident:{id}` and its duration comes from a single `CLAIM_TTL` constant (60 seconds) shared by every site that acquires, renews or reports a claim — a duplicated and later divergent TTL would make locks expire sooner than the clients believe. Acquisition uses a single `SET key value NX PX CLAIM_TTL` command, which is atomic by definition (Redis is single-threaded).

A failed `SET NX` does not immediately imply a conflict, because the existing lock may belong to the caller: the handler therefore falls through to `RENEW_LOCK_SCRIPT`, and only a lock owned by *somebody else* yields `409`. Claiming is thus **idempotent for the current holder**, who re-acquires and extends their own claim instead of being locked out of it. Renewal goes through the Lua script rather than a bare `PEXPIRE` for the usual reason: between a check and an unconditional extension the lock could expire and be taken by another analyst, and the renewal would then prolong *their* claim.

Release uses a Lua script (`RELEASE_LOCK_SCRIPT`) that conditionally deletes the key only if the stored value still matches the releasing operator, preventing a late release from accidentally removing someone else's subsequently-acquired lock. The

same script governs the release performed inside the `PATCH` handler, for every target status rather than only for `closed`.

- **Optimistic locking (MongoDB version field):** used for incident resolution. Every incident document carries a `version` integer, initialized to 1 on creation and incremented atomically on every update via `$inc: {version: 1}`. The `PATCH` handler includes the client-supplied `version` in the query filter of the `updateOne` call; if the version has been incremented since the client last read the document (indicating a concurrent modification), the `matchedCount` is 0 and the request is rejected with HTTP 409 without modifying the document.

4.6.6 Leader Election Implementation

The background escalation task must execute exactly once per cycle cluster-wide, regardless of how many `api` pods are running. Leadership is implemented as a Redis key (`leader:escalation`) with a 15-second TTL:

- Every 10 seconds, each pod attempts `SET leader:escalation <podId> NX PX 15000`. If no leader exists, one pod succeeds and becomes leader.
- If the `SET NX` fails (another pod is already leader), the pod attempts to *renew* the lock using the `RENEW_LOCK_SCRIPT` Lua script, which atomically verifies ownership and extends the TTL only if the stored value matches the current pod's identifier. This prevents the split-brain scenario where a non-atomic `GET+PEXPIRE` would allow a pod to extend a lock that has already expired and been acquired by another pod in the interval between the two commands.
- Only the pod that holds (or has just acquired) the leadership key executes the escalation query (`updateMany` on stale open incidents). If the leader pod crashes, the key expires after at most 15 seconds and another pod takes over on the next 10-second cycle.

4.6.7 Real-Time Propagation Implementation

Two propagation channels coexist, deliberately separated based on whether the originating change is persisted in MongoDB:

- **Persistent changes** (insert, update of incident documents) are propagated via **MongoDB Change Streams**. Each `api` pod opens its own independent change-stream cursor (`db.collection('incidents').watch([], {fullDocument: 'updateLookup'})`) and emits the change event to its own locally-connected clients via `io.local.emit('incident.update.change')`. The `local` modifier is critical: since every pod independently receives the same change event from MongoDB, using `io.emit()` would cause the event to also cross pod boundaries through the Redis adapter, resulting in duplicate delivery to clients connected to other pods.

- **Ephemeral changes** (lock acquired, lock released, presence changed) are propagated via **Socket.IO broadcasts through the Redis adapter**. Since only the one pod that handled the originating REST request knows the change happened (no MongoDB document was modified), `io.emit()` is used, which the Redis adapter propagates to all pods, ensuring all connected clients are notified regardless of which pod they are connected to.

An `incident_unlocked` broadcast is emitted only when `RELEASE_LOCK_SCRIPT` reports that a lock was genuinely deleted, never unconditionally after a release attempt. Emitting it regardless was a defect in an earlier iteration: if the caller’s claim had already expired and been re-acquired by a different analyst, the broadcast instructed every client to clear `lockedBy` while Redis still held a perfectly valid lock for the new owner, who consequently lost the action buttons on a ticket they legitimately held.

Neither channel can announce a lock’s *natural expiry*, since no pod is involved when a TTL elapses; that case is handled client-side (section 4.6.8).

4.6.8 Client-Side State Management

The Vue.js client maintains a single reactive `incidents` array that serves as the client’s local copy of the system state. This array is:

- **Hydrated** on every (re)connection, and on every `resync_required`, by fetching `GET /api/incidents`. Each fetch increments a *resynchronization-generation* counter and applies its result only if it is still the most recent one, so that a slow response can never overwrite a fresher snapshot (NFR7). The counter is per-resynchronization rather than per-connection deliberately: two hydrations can now be in flight within a single connection (a reconnect immediately followed by a server-requested resync), and a per-connection counter would let both pass the staleness check;
- **Incrementally updated** by the five Socket.IO event handlers (`incident_update`, `incident_locked`, `incident_unlocked`, `users_update`, `resync_required`);
- **Locally expired** for claims: every lock the client learns about — whether from a hydration (`lockTtl`) or from an `incident_locked` event (`ttl`) — schedules a timer that clears `lockedBy` once the TTL elapses, unless a newer event for that incident has replaced the timer in the meantime. The timer is the client’s only way to observe an expiry, since Redis lets the key vanish without telling anyone (section 3.4); it verifies that the owner is still the one it was scheduled for before clearing, so a claim renewed or re-acquired in the interim is not erased. All timers are discarded on hydration and on logout, so they can never outlive the state they describe;
- **Filtered** through a chain of computed properties (`filteredIncidents` → `kanbanOpen`, `kanbanInProgress`, `kanbanEscalated`, `kanbanResolved`) that react to search text,

the “escalated only” toggle, and the “mine only” toggle. Critically, the KPI counters (`totalIncidents`, `openIncidents`, etc.) read from the unfiltered `incidents` array, so they always reflect the true global state regardless of active filters (FR9).

The `incident_update` handler merges persistent state changes (from the Change Stream) with ephemeral lock state (from `incident_locked/incident_unlocked`) by preserving the current `lockedBy` value when updating a document. This prevents the race condition described in NFR9, where a lock-release event and a database change-stream event for the same incident could arrive in either order and, without this merge logic, cause the incident to appear in two columns simultaneously.

5 Validation

This section describes how each requirement from section 2 was validated against the acceptance criteria defined there.

5.1 Automatic Testing

Unit testing. No unit-test framework is configured for either the API or the front-end component (the `package.json` test script is the default `echo "Error: no test specified"`). This is a deliberate scoping choice for a single-developer project: the API’s domain logic is minimal (the business rules are almost entirely expressed as database queries and Redis commands), and extracting it into independently-testable units would require introducing a repository/service abstraction layer that the codebase does not currently have. Given the project’s scope, the effort of creating that abstraction solely for testability was deemed disproportionate to the benefit.

Integration and end-to-end testing. The properties that matter most in this system — mutual exclusion under concurrency, leader election correctness, fault tolerance under pod failure, real-time propagation across multiple clients — are inherently *integration-level* and *infrastructure-dependent* properties. They cannot be meaningfully tested without a running MongoDB Replica Set, a live Redis instance, multiple API pods, and an Ingress routing real traffic, which is precisely the production-like environment that Kubernetes provides.

For this reason, **the Kubernetes deployment itself serves as the test harness**: every test is executed against the full system running on a Minikube cluster, with real inter-component communication, real concurrency, and real failure injection via `kubectl delete pod`. This approach exercises the exact code paths that would run in production, including Kubernetes-level mechanisms (readiness probes, PodDisruptionBudgets, HPA) that a mocked test environment would not exercise.

Reproducibility. All tests can be reproduced on any machine with Minikube, Docker, and `kubectl` installed, by following the deployment instructions and then executing the `kubectl` and `curl` commands listed below.

5.2 Acceptance Tests

All validation was performed via **manual testing** on the live Kubernetes cluster. Each test is mapped to the specific requirement(s) it validates and describes the procedure, expected outcome, and observed result.

Validation status. The results reported below were observed on the live cluster. Where a test's procedure has since been *extended* to cover behaviour introduced by the last round of corrections — the claim state carried in the board snapshot, the idempotent re-claim, the client-side expiry of a lock, and the Change Stream's recovery path — the additional steps are stated as expectations to be checked rather than as observations, and are marked as such; the pre-existing steps and their results are unchanged.

Why manual. Automating these tests would require an end-to-end testing framework capable of: (i) orchestrating multiple browser sessions simultaneously, (ii) injecting pod failures via the Kubernetes API, (iii) observing real-time WebSocket events across clients, and (iv) waiting for asynchronous infrastructure-level responses (readiness probe recovery, HPA scaling decisions). Tools like Cypress or Playwright could cover the browser-side scenarios, but integrating them with `kubect1`-driven failure injection and Kubernetes event observation was beyond the time budget of this single-developer project. The manual tests documented here cover the same acceptance criteria and are fully reproducible.

5.2.1 FR1 — Board Access

- **Procedure:** open `http://board.local` in a browser; register a new account (username “alice”, a password of at least 8 characters) and, once redirected to the login form, log in with the same credentials.
- **Expected:** on successful login, the screen transitions to the Kanban dashboard; the browser's developer tools show an active, authenticated WebSocket connection to `/socket.io` (the JWT is present in `auth.token` of the handshake); the “Users Online” KPI shows at least 1. Logging in with a wrong password, or attempting to register an already-taken username, is rejected with an error message and no dashboard access.
- **Result:** passed. The submit button stays disabled until both the username and password fields are non-empty (client-side guard); the server independently rejects short passwords, duplicate usernames, and wrong credentials regardless of what the client sends.

5.2.2 FR2 — Incident Creation

- **Procedure:** from the dashboard, type a title in the “Register New Incident” field and click “Submit”. Alternatively, send a direct REST call authenticated as a previously registered user (TOKEN obtained from `POST /api/auth/login`):

```
curl -X POST http://board.local/api/incidents \
-H "Authorization: Bearer $TOKEN" \
-H "Content-Type: application/json" \
-d '{"title":"Test incident","status":"open"}'
```

The `createdBy` field is no longer accepted from the request body: the server derives it from the authenticated username carried by `TOKEN`.

- **Expected:** the API returns 201; the new incident appears in the “Open” column of the originating client *and* of a second client connected in a different browser, without either client refreshing.
- **Result:** passed. The Change Stream triggers an `incident_update` event of type `insert`, which both clients receive independently.

5.2.3 FR3 — Real-Time Board Synchronization

- **Procedure:** open two browser windows as “alice” and “bob”. From alice’s window, create an incident, claim it, and resolve it. Observe bob’s window after each step.
- **Expected:** each action (creation, claim, resolution) is reflected on bob’s board within approximately one second, with no manual refresh or polling.
- **Result:** passed. Creation and resolution are propagated via the Change Stream (`incident_update`); the claim is propagated via the direct Socket.IO broadcast (`incident_locked`).

5.2.4 FR4 — Ticket Claiming (Mutual Exclusion)

- **Procedure:** create an incident, then attempt to claim it from two authenticated clients simultaneously (`ALICE_TOKEN` and `BOB_TOKEN` obtained from separate `POST /api/auth/login` calls):

```
curl -X POST http://board.local/api/incidents/<id>/claim \
-H "Authorization: Bearer $ALICE_TOKEN" &
curl -X POST http://board.local/api/incidents/<id>/claim \
-H "Authorization: Bearer $BOB_TOKEN" &
wait
```

The lock owner is no longer taken from a `username` field in the request body: it is the authenticated username carried by each token, so neither client can claim on behalf of the other.

- **Expected:** exactly one request returns 200 with `{"success":true, "lockedBy":"...", "ttl":60000}`; the other returns 409 with the name of the current owner.
- **Result:** passed. The Redis `SET NX PX` command is atomic; concurrent arrivals are serialized by Redis's single-threaded execution.
- **Additional check (idempotency for the holder).** Repeat the claim with `ALICE_TOKEN` while alice still holds the lock: the expected answer is 200 rather than 409, with the lock's remaining TTL (`redis-cli PTTL lock:incident:<id>` inside the Redis pod) back at approximately 60000 ms. Repeating it with `BOB_TOKEN` must still return 409, since the fall-through renewal path is reachable only by the current owner and mutual exclusion is therefore unaffected.

5.2.5 FR5 — Explicit Claim Release

- **Procedure:** claim an incident as “alice”, then click “Release” (or send `DELETE /api/incidents/<id>/claim` authenticated as alice, i.e., with `-H "Authorization: Bearer $ALICE_TOKEN"` and no body — the releasing username is taken from the token, not from a request field). Immediately attempt to claim the same incident as “bob”.
- **Expected:** the release returns 200; bob's subsequent claim succeeds with 200; the incident moves back from “In Progress” to its original column (“Open” or “Escalated”) on all clients.
- **Result:** passed. The `RELEASE_LOCK_SCRIPT` atomically verifies ownership before deleting the key.

5.2.6 FR6 — Ticket Resolution

- **Procedure:** claim an incident as “alice”, then click “Resolve” (or send `PATCH /api/incidents/<id>` authenticated as alice with `{"status":"closed", "version":<current>}` — `closedBy` is no longer accepted as a request field and is instead taken from the authenticated username).
- **Expected:** the API returns 200; the incident moves to the “Resolved” column on all clients; the “Claim” button is no longer available for this incident; the `closedBy` field shows “alice”.
- **Result:** passed. The lock is released atomically by the `RELEASE_LOCK_SCRIPT` within the same `PATCH` handler, for any target status rather than only for `closed`, and the `incident_unlocked` broadcast is emitted only if the script reports that a lock was actually deleted.

5.2.7 FR7 — Automatic SLA Escalation

- **Procedure:** create an incident and leave it in `open` status for more than 5 minutes. Monitor the Kanban board and the API pod logs (`kubect1 logs -l app=api -n app --tail=50`).
- **Expected:** the incident transitions to `escalated` within ≤ 10 seconds after the 5-minute threshold, moves to the “Escalated” column on all clients, and a toast notification appears. The pod logs show exactly one escalation log entry (`[LEADER <podId>] Escalati automaticamente 1 incidenti...`).
- **Result:** passed. The background task fires every 10 seconds; the Change Stream propagates the status change to all clients.

5.2.8 FR8 — Presence Awareness

- **Procedure:** open two browser tabs as “alice” and “bob”. Observe the “Users Online” KPI. Close bob’s tab.
- **Expected:** with both tabs open, the KPI shows 2; after closing bob’s tab, it drops to 1 within a few seconds.
- **Result:** passed. The `io.fetchSockets()` call on the `disconnect` event retrieves only the remaining live sockets.

5.2.9 FR9 — Filtering and Search

- **Procedure:** with several incidents in different states, type a partial title in the search bar; toggle “Mostra solo Escalated”; toggle “I Miei Ticket”. Observe the Kanban columns and the KPI counters.
- **Expected:** the Kanban columns show only the matching incidents; the KPI counters (Total, Open, Escalated, Resolved) remain unchanged and continue to reflect the full, unfiltered system state.
- **Result:** passed. The KPI `computed` properties read from the raw `incidents` array, while the Kanban columns read from the derived `filteredIncidents`.

5.2.10 FR10 — Event Notifications

- **Procedure:** trigger the following events and observe the toast notifications:
 - create an incident from client A → client B receives an info toast (“Nuovo ticket registrato: ...”);
 - resolve an incident from client A → client B receives a success toast (“Il ticket ... è stato risolto da ...”);
 - attempt to claim an already-claimed incident → the requesting client receives an error toast (“Impossibile prendere in carico: Incidente già preso in carico”);

- attempt to resolve with a stale version → the requesting client receives an error toast (“Conflitto o errore: ...”).
- **Expected:** every event produces a visible, non-blocking toast notification with a descriptive message; no action fails silently.
- **Result:** partially passed. The `try/catch` blocks in `claimIncident`, `releaseClaim`, and `resolveIncident` surface failures via `toast.error()`, including the defensive parsing of non-JSON error bodies. `createIncident`, however, neither wraps its `fetch` call in a `try/catch` block nor checks `response.ok`: a failed creation request still displays the success toast, silently violating this requirement for that one action. This gap was identified while writing this report and is left unfixed as a known limitation (section 3.7).

5.2.11 NFR1 — No Lost Updates (Optimistic Locking)

- **Procedure:** claim an incident (version 1) in two separate `curl` sessions. From the first session, resolve it (sending `version:1`). From the second session, attempt to resolve it again (still sending `version:1`).
- **Expected:** the first resolution succeeds (200); the second is rejected with 409 (“Conflitto! Il documento è stato già modificato o non esiste.”) because the stored version is now 2.
- **Result:** passed.

5.2.12 NFR2 — Server-Enforced Mutual Exclusion

- **Procedure:** claim an incident as “alice” via the UI. Then, bypassing the UI entirely, send a direct PATCH request authenticated as “bob” (a genuinely registered, logged-in user, i.e., a real `BOB_TOKEN` from `POST /api/auth/login`):

```
curl -X PATCH http://board.local/api/incidents/<id> \
-H "Authorization: Bearer $BOB_TOKEN" \
-H "Content-Type: application/json" \
-d '{"status":"closed","version":1}'
```

- **Expected:** the API rejects the request with 409 (“Incidente bloccato da un altro utente”, `lockedBy: "alice"`), even though the UI would have hidden the resolve button for bob, and even though bob is authenticating with his own genuine credentials rather than a spoofed identity.
- **Result:** passed. The server-side check (`currentOwner && currentOwner !== req.user.username`) compares the lock owner against bob’s identity as derived from his verified JWT, not from any client-supplied field — so this check can no longer be defeated even by a client that tries to claim a different `closedBy` in the request body, since that field is ignored entirely.

5.2.13 NFR3 — Fault Tolerance of the API Tier

- **Procedure:** verify that ≥ 3 API pods are running (`kubectl get pods -n app -l app=api`). Delete one pod (`kubectl delete pod <name> -n app`). Immediately attempt to create and claim an incident from the browser.
- **Expected:** the operations succeed without visible downtime; the remaining pods continue serving requests; the deleted pod is automatically recreated by the Deployment controller; Socket.IO clients that were connected to the terminated pod reconnect transparently (the sticky-session cookie is rebalanced to a surviving pod).
- **Result:** passed. Kubernetes recreated the pod within seconds; the Ingress redirected traffic to the surviving pods; the Socket.IO client's automatic reconnection fired the `connect` event, re-fetching the incident list.
- **Additional check (Change Stream as a readiness dependency).** A pod whose Change Stream has died is no longer considered a healthy backend. Expected behaviour: `GET /api/ready` on that pod answers 503 with `"change stream non attivo"`, Kubernetes removes it from the Service's endpoint list without restarting it (the liveness probe, being shallow, keeps passing), its clients reconnect to a ready pod, and the affected pod rejoins once its cursor is rebuilt. The log line `Change stream interrotto` followed by `Change stream ripristinato` on the same pod (`kubectl logs`) marks the two transitions. Injecting a genuinely non-resumable stream failure from outside the process is not straightforward — the driver transparently retries the resumable ones, which is precisely why the escalating failure scenarios used elsewhere in this section (deleting a `mongo` pod) do *not* exercise this path — so this check has not been reproduced on the cluster and is documented as the intended behaviour of the code rather than as an observed result.

5.2.14 NFR4 — Fault Tolerance of the Data Tier

- **Procedure:** identify the current MongoDB primary (`kubectl exec mongo-0 -n app -- mongosh --quiet --eval "rs.isMaster().primary"`). Delete the primary pod (`kubectl delete pod mongo-0 -n app`). Monitor the replica-set status from a secondary (`kubectl exec mongo-1 -n app -- mongosh --quiet --eval "rs.status().members.map(m => m.stateStr)"`).
- **Expected:** within 10–30 seconds, the remaining two members elect a new primary; the API pods' readiness probes may briefly fail (returning 503), causing Kubernetes to temporarily remove them from the Service's endpoint list; once the new primary is available, the readiness probes pass again and the API resumes normal operation. The deleted `mongo-0` pod is recreated by the StatefulSet controller and re-joins the replica set as a secondary.

- **Result:** passed. The PodDisruptionBudget (`minAvailable: 2`) prevented involuntary simultaneous eviction; the election completed within approximately 15 seconds.

5.2.15 NFR5 — Exactly-Once Background Execution

- **Procedure:** create an incident and wait for it to be escalated. Observe the combined logs of all API pods:

```
kubectl logs -l app=api -n app --tail=100 | grep "Escalati"
```

- **Expected:** exactly one log line reports the escalation, from exactly one pod (the leader). No duplicate escalation appears, and the incident's `version` is incremented by exactly 1.
- **Result:** passed. The `SET NX PX + RENEW_LOCK_SCRIPT` pattern ensures that only one pod holds the `leader:escalation` key at any time.

5.2.16 NFR6 — Horizontal Scalability (HPA)

- **Procedure:** check the current number of API pods (`kubectl get hpa -n app`). Generate sustained CPU load on the API pods (e.g., a tight loop of `curl` requests). Monitor the HPA's scaling decision (`kubectl get hpa api-hpa -n app -w`).
- **Expected:** when the average CPU utilization exceeds 80%, the HPA increases the replica count (up to the configured maximum of 10) within its stabilization window.
- **Result:** passed (observed scale-up from 3 to 5 replicas under artificial load).

5.2.17 NFR7 — Reconnection Consistency

- **Procedure:** with a client connected, perform a rolling restart of the API deployment (`kubectl rollout restart deployment/api -n app`). Observe the client's behavior during the rapid reconnection cycle.
- **Expected:** the client reconnects automatically; the Kanban board displays the current state, never an older snapshot from a superseded connection.
- **Result:** passed. The resynchronization-generation counter in the Vue client discards stale fetch responses. During the rolling restart, the client experienced 1–2 brief disconnections (visible via the `Socket.IO` reconnection in the browser console) but always converged to the correct state.

- **Additional check (claim survives a reload).** Claim an incident as alice, then reload alice’s browser tab without releasing it. Expected: the card is still in “In Progress” showing “Locked by: alice”, the **Resolve/Release** buttons are still available to alice, and the response of `GET /api/incidents` contains `lockedBy` and a `lockTtl` lower than 60000 for that incident. Then leave the tab idle without acting: when the TTL elapses, the card returns on its own to “Open” (or “Escalated”) on every client, with no server event involved.

5.2.18 NFR8 — Self-Healing Presence Data

- **Procedure:** connect two clients (“alice” and “bob”); verify the “Users Online” KPI shows 2. Force-terminate one API pod (`kubectl delete pod <pod-serving-bob> -n app --force --grace-period=0`) to simulate an unclean shutdown (no `SIGTERM`, no `disconnect` event).
- **Expected:** the “Users Online” count converges to the correct value (1, if bob’s browser has not yet reconnected to another pod; 2, if it has) without any manual cleanup of stale Redis state.
- **Result:** passed. Since presence is derived from `io.fetchSockets()` (live socket introspection via the Redis adapter, which relies on `Socket.IO`’s `engine.io` heartbeat to detect dead connections), stale sessions are automatically pruned when the heartbeat timeout expires. The earlier implementation, which maintained a manual Redis hash (`active_users`), failed this test (stale entries persisted indefinitely), which motivated the switch to `fetchSockets()`.

5.2.19 NFR9 — UI Eventual Consistency

- **Procedure:** claim an incident as “alice” (it moves to “In Progress”). Resolve it. Observe whether the incident briefly appears in both “In Progress” and “Resolved” columns simultaneously.
- **Expected:** the incident transitions atomically from one column to the other, never appearing in two columns at once.
- **Result:** passed. The `incident_update` handler preserves the current `lockedBy` value when merging a Change Stream update, and the `incident_unlocked` handler clears it independently; since the Kanban columns are derived from `computed` properties that check both `status` and `lockedBy`, a transient intermediate state (e.g., status changed to `closed` but lock not yet released) is classified into exactly one column, not two.

6 Deployment

This section provides a complete, reproducible procedure to deploy the system from scratch on a local machine using Minikube. No cloud provider account or remote registry

is required.

6.1 Prerequisites

The following software must be installed before proceeding. The versions listed are those used during development; newer patch releases are expected to work.

- **Docker Desktop** (or Docker Engine) $\geq 24.x$ — required to build the container images.
- **Minikube** ≥ 1.33 — provides the single-node Kubernetes cluster. The **docker** driver is recommended.
- **kubectl** ≥ 1.30 — the Kubernetes command-line client. Must be compatible with the cluster version that Minikube provisions.
- **Node.js** ≥ 20 and **npm** ≥ 10 — only needed if one wishes to run the API or the frontend outside of containers (e.g., during local development); not required for the Kubernetes deployment itself.

6.2 Cluster Setup

Step 1: Start Minikube. Create a local cluster with the Ingress addon enabled:

```
minikube start --driver=docker
minikube addons enable ingress
minikube addons enable metrics-server    # required for the HPA
```

The **ingress** addon deploys an Nginx Ingress Controller inside the cluster. The **metrics-server** addon is necessary for the **HorizontalPodAutoscaler** to read CPU utilization metrics.

Step 2: Create the namespace. All project resources are deployed in the **app** namespace:

```
kubectl create namespace app
```

Step 3: Configure /etc/hosts. The Ingress routes traffic based on the hostname **board.local**. Add the following entry to the host machine's **/etc/hosts** file (requires administrator privileges):

```
<minikube-ip> board.local
```

where **<minikube-ip>** is the output of **minikube ip**. On macOS with the **docker** driver, a **minikube tunnel** process must be running in a separate terminal (in this case, use 127.0.0.1 as the IP).

6.3 Building the Container Images

Because the Kubernetes manifests set `imagePullPolicy: Never`, the images must be built *inside* Minikube's Docker daemon, not the host's. This is achieved by pointing the local shell's Docker client at Minikube's daemon:

```
eval $(minikube docker-env)
```

Then, from the project root:

```
docker build -t api:latest ./api
docker build -t web:latest ./web
```

Expected outcome: two images (`api:latest`, `web:latest`) are available inside Minikube's Docker daemon. They can be verified with `docker images | grep -E 'api|web'`.

API image (`api/Dockerfile`): a single-stage build from `node:20-alpine`. It copies `package*.json` first, runs `npm ci --only=production` to install dependencies (leveraging Docker's layer cache for faster rebuilds), then copies the application code. The entrypoint is `node index.js`, listening on port 3000.

Web image (`web/Dockerfile`): a multi-stage build. Stage 1 (`node:20-alpine`) installs all dependencies (including dev dependencies needed by Vite) and runs `npm run build`, producing the compiled Vue 3 SPA in `/app/dist`. Stage 2 copies only the `dist/` directory into an `nginx:alpine` image, yielding a minimal production image that serves static files on port 80 with no Node.js runtime.

Third-party images. MongoDB (`mongo:6`) and Redis (`redis:7`) use the official upstream images directly and are pulled automatically by Kubernetes on first deployment (they do *not* have `imagePullPolicy: Never`).

6.4 Deploying the Kubernetes Manifests

The manifests in `k8s/` must be applied in a specific order to respect inter-component dependencies. Each step includes the command, the resources it creates, and the expected outcome.

Step 1: MongoDB (StatefulSet, Headless Service, ConfigMap, Init Job, PDB).

```
kubectl apply -f k8s/mongo.yaml
```

This creates five resources: the headless Service (`mongo`), the ConfigMap (`mongo-init`) containing the `rs.initiate()` script, the StatefulSet with 3 replicas, the initialization Job (`mongo-rs-init`), and the PodDisruptionBudget (`mongo-pdb`).

Expected outcome: the three `mongo` pods start in order (`mongo-0`, `mongo-1`, `mongo-2`); the init Job waits for all three to become reachable, then initializes the Replica Set `rs0` and waits for a primary election. This step takes approximately 60–90 seconds. Progress can be monitored with:

```
kubectl get pods -n app -l app=mongo -w
kubectl logs job/mongo-rs-init -n app -f
```

Step 2: Redis (Deployment, Service).

```
kubectl apply -f k8s/redis.yaml
```

Expected outcome: a single `redis` pod starts and becomes ready within a few seconds.

Step 3: JWT Secret. The `api` Deployment requires the `jwt-secret` Secret to exist before it starts, since `JWT_SECRET` is injected via `secretKeyRef`:

```
kubectl apply -f k8s/jwt-secret.yaml
```

Expected outcome: the Secret `jwt-secret` is created in the `app` namespace, holding the HMAC key shared by all `api` replicas. For any deployment beyond local development/demo purposes, the placeholder value committed in the manifest should be replaced, e.g. by regenerating it with `kubectl create secret generic jwt-secret --namespace app --from-literal=JWT_SECRET=$(openssl rand -base64 48) --dry-run=client -o yaml | kubectl apply -f -`.

Step 4: API (Deployment, Service).

```
kubectl apply -f k8s/api.yaml
```

Expected outcome: 3 `api` pods start. Each pod connects to the MongoDB Replica Set and to Redis on startup, and reads the shared `JWT_SECRET` from the Secret created in Step 3; if any dependency is not yet available, the pod exits with a fatal error and Kubernetes restarts it (the readiness probe prevents it from receiving traffic until all dependencies are reachable). Steady state is reached within 20–30 seconds:

```
kubectl get pods -n app -l app=api
```

All 3 pods should show `Running` with `READY 1/1`.

Step 5: HorizontalPodAutoscaler.

```
kubectl apply -f k8s/hpa.yaml
```

Expected outcome: the HPA `api-hpa` is created, targeting the `api` Deployment with a 3–10 replica range and an 80% CPU utilization target. The `metrics-server` addon must be running for the HPA to function:

```
kubectl get hpa -n app
```

Step 6: Web frontend (Deployment, Service).

```
kubectl apply -f k8s/web.yaml
```

Expected outcome: a single `web` pod starts, running the Nginx container that serves the compiled Vue 3 SPA on port 80.

Step 7: Ingress.

```
kubectl apply -f k8s/ingress.yaml
```

Expected outcome: the Ingress resource `app-ingress` is created with three path-based routing rules (`/` → `web:80`, `/api` → `api:3000`, `/socket.io` → `api:3000`) and cookie-based sticky sessions enabled. After a few seconds, `kubectl get ingress -n app` should show an `ADDRESS` matching the Minikube IP.

Verification. Once all manifests are applied, the full deployment can be verified in a single command:

```
kubectl get all -n app
```

Expected output should show: 3 `api` pods (Running), 3 `mongo` pods (Running), 1 `redis` pod (Running), 1 `web` pod (Running), 1 completed Job (`mongo-rs-init`), plus their respective Services, Deployments, StatefulSet, and ReplicaSets.

Opening `http://board.local` in a browser should display the login screen. On macOS with the `docker` driver, `minikube tunnel` must be running.

6.5 Environment Variables

The `api` pods receive their configuration via environment variables defined in `k8s/api.yaml`. Most values are set directly in the manifest; the one sensitive value, `JWT_SECRET`, is instead injected from a dedicated Kubernetes `Secret` (`k8s/jwt-secret.yaml`) via `secretKeyRef`, so that it never appears in plain text in the Deployment spec itself. No `.env` files or ConfigMaps are used for the API configuration.

- `MONGO_URL` — the MongoDB connection string, listing all three Replica Set members with their stable StatefulSet DNS names and the `replicaSet=rs0` parameter:

```
mongodb://mongo-0.mongo:27017,mongo-1.mongo:27017,  
mongo-2.mongo:27017/app?replicaSet=rs0
```

This format enables the MongoDB driver to perform automatic primary discovery and failover-transparent reconnection.

- `REDIS_URL` — the Redis connection endpoint, using the ClusterIP Service name: `redis://redis:6379`.

- `CORS_ORIGIN` — the allowed origin for CORS and Socket.IO cross-origin requests: `http://board.local`, matching the Ingress hostname.
- `JWT_SECRET` — the HMAC secret used to sign and verify JSON Web Tokens, sourced from the `jwt-secret` Kubernetes `Secret` rather than a literal value. All three `api` replicas mount the same `Secret`, which is what allows any pod to verify a token issued by any other pod without shared session state or sticky sessions.

Additionally, the API code reads `os.hostname()` to obtain a unique pod identifier (e.g., `api-7685fc6bd-4wv68`), used as the value for Redis leader-election and lock keys. This requires no explicit configuration, as Kubernetes automatically sets the hostname to the pod name.

The `web` and `redis` deployments require no environment variables. The `mongo` StatefulSet receives its configuration via command-line arguments (`--replSet rs0 --bind_ip_all`) rather than environment variables.

6.6 Deployment Architecture

The project does **not** use Docker Compose. While Docker Compose would be simpler for local development, it cannot exercise the Kubernetes-specific distributed-systems mechanisms that are central to the project’s educational objectives: StatefulSets with stable network identities, PodDisruptionBudgets, HorizontalPodAutoscaler, Ingress with session affinity, and liveness/readiness probe separation. The deployment is therefore defined entirely as Kubernetes manifests (`k8s/*.yaml`), validated on Minikube.

The seven manifest files and their purposes are summarized below:

- `mongo.yaml` — 5 resources: headless Service, ConfigMap (init script), StatefulSet (3 replicas with PVCs), init Job, PodDisruptionBudget.
- `redis.yaml` — 2 resources: Deployment (1 replica), ClusterIP Service.
- `api.yaml` — 2 resources: Deployment (3 replicas, liveness/readiness probes, resource limits), ClusterIP Service.
- `jwt-secret.yaml` — 1 resource: `Secret` holding `JWT_SECRET`, the HMAC key shared by all `api` replicas to sign and verify authentication tokens.
- `hpa.yaml` — 1 resource: HorizontalPodAutoscaler (3–10 replicas, 80% CPU target).
- `web.yaml` — 2 resources: Deployment (1 replica), ClusterIP Service.
- `ingress.yaml` — 1 resource: Ingress (host `board.local`, 3 path rules, cookie-based sticky sessions).

Redeployment after code changes. After modifying the API or frontend source code, the corresponding Docker image must be rebuilt inside Minikube’s Docker daemon and the Deployment must be restarted to pick up the new image:

```
eval $(minikube docker-env)
docker build -t api:latest ./api          # or ./web
kubectl rollout restart deployment/api -n app # or web
```

The rolling restart ensures zero-downtime redeployment: Kubernetes creates new pods with the updated image before terminating the old ones, and the readiness probe prevents traffic from reaching a pod until it has fully initialized.

7 User Guide

This section describes how to use the application from the operator’s perspective, assuming the system has been deployed and is reachable at <http://board.local>.

7.1 Accessing the Board

1. Open <http://board.local> in a modern web browser (Chrome, Firefox, Edge, or Safari).
2. The **login screen** is displayed: a centered card with a pulsing green indicator, a title (“Accedi alla Board”), a username field, and a password field.
3. **First-time operators** click the “Registrati” link to switch the card to registration mode, enter a username and a password of at least 8 characters, and submit. On success, the form switches back to login mode.
4. **Returning operators** type their username and password directly and either press **Enter** or click the “Accedi” button. The button remains disabled until both fields are non-empty. An incorrect username/password combination is rejected with an inline error message and the operator remains on the login screen.
5. **Expected outcome:** on a successful login, the screen transitions to the main dashboard. A success toast notification confirms the connection (“Benvenuto nella dashboard, alice!”). A persistent, authenticated WebSocket connection is established in the background, and a “Esci” (logout) button appears in the header to end the session.

7.2 Dashboard Overview

The dashboard is organized vertically into four rows:

Row 1: KPI Summary Cards. Five cards display real-time aggregate counters:

- **Total Tickets** — the total number of incidents in the system (all statuses).
- **Open Tickets** — incidents with status `open` that have not been claimed.
- **Escalated Tickets** — incidents automatically escalated for SLA breach.
- **Resolved Tickets** — incidents marked as `closed`.
- **Users Online** — the number of operators currently connected to the board.

These counters always reflect the *true*, *unfiltered* global state, regardless of any active search or filter.

Row 2: Chart and Incident Creation.

- On the left, a **bar chart** (“Incidents Status Overview”) displays the distribution of incidents across Open, Escalated, and Resolved statuses. The chart updates reactively as incidents change state. If no incidents exist, a “No data available” placeholder is shown.
- On the right, a **creation form** (“Register New Incident”) allows the operator to type an incident title and submit it via the “Submit” button or the **Enter** key.

Row 3: Filters and Search. A horizontal bar containing:

- A **search field** (“Cerca per titolo dell’incidente...”) that filters incidents by title in real time (case-insensitive substring match).
- An **“Escalated only” toggle** (“Mostra solo Escalated”) that, when active (highlighted in orange), restricts the Kanban board to show only escalated incidents.
- A **“My tickets” toggle** (“I Miei Ticket”) that, when active (highlighted in blue), restricts the board to incidents where the current operator is the creator, the current lock holder, or the resolver.

All three filters can be combined. Toggling a filter again deactivates it. See section 7.8 for details.

Row 4: Kanban Board. Four columns represent the incident lifecycle:

- **Open** (magenta header) — unclaimed incidents with status `open`. Each card shows the title, the creator’s name, and a “Claim Ticket” button.
- **In Progress** (blue header) — incidents currently claimed (locked) by an operator, regardless of their underlying status (`open` or `escalated`). Each card shows the title, an “ESCALATED” badge if applicable, and the lock holder’s name. If the current operator holds the lock, “Resolve” and “Release” buttons are displayed.

- **Escalated** (orange header) — unclaimed incidents with status `escalated`. Each card shows the title, an SLA warning, and a “Claim Escalation” button.
- **Resolved** (green header) — incidents with status `closed`. Each card shows the title (struck through) and the resolver’s name. No action buttons are available.

Each column header includes a badge showing the current count of cards in that column. Cards appear and disappear with a smooth vertical slide animation.

7.3 Creating an Incident

1. In the “Register New Incident” card, type a descriptive title for the incident (e.g., “Database connection timeout on prod-db-03”).
2. Click “Submit” or press **Enter**. The button is disabled if the field is empty.
3. **Expected outcome:** a success toast appears (“Incidente registrato con successo!”). The new incident appears as a card in the “Open” column on the current client and on *every other connected client* simultaneously, without any manual refresh. The “Total Tickets” and “Open Tickets” KPI counters increment by 1. The input field is cleared, ready for the next entry.

7.4 Claiming an Incident

Claiming an incident grants the operator exclusive access to it, preventing other operators from claiming or resolving it simultaneously.

1. Locate the incident in the “Open” or “Escalated” column.
2. Click the “Claim Ticket” (or “Claim Escalation”) button on the card.
3. **Expected outcome (success):** a success toast appears (“Hai preso in carico il ticket.”). The incident card moves from the “Open” (or “Escalated”) column to the “In Progress” column. The card now shows “Locked by: <your username>” and displays “Resolve” and “Release” buttons. On all other clients, the same card moves to “In Progress” and shows the lock holder’s name, but without the action buttons (only the lock holder sees them).
4. **Expected outcome (conflict):** if another operator has already claimed the incident, an error toast appears (“Impossibile prendere in carico: Incidente già preso in carico”) and reports who currently holds the lock. No state change occurs.

The claim expires automatically after 60 seconds if the operator does not resolve or explicitly release it. At that moment the card returns on its own to “Open” (or “Escalated”) on every client — including the holder’s — and the incident becomes claimable again by anyone.

Two consequences are worth knowing in practice:

- **Reloading the page does not lose the claim.** After a refresh (or a reconnection following a pod restart) the incident is still shown as “In Progress” with the operator’s own name and their **Resolve/Release** buttons, because the claim is part of the state the client reloads from the server, not merely something it was told about once.
- **Clicking “Claim” on a ticket one already holds is harmless, and useful.** It does not produce a conflict: it renews the claim for another 60 seconds. This is the way to keep a lengthy investigation from having its claim expire underneath it.

7.5 Releasing a Claim

If an operator has claimed an incident but decides not to resolve it (e.g., the issue was misassigned or the operator needs to leave), they can release the lock explicitly.

1. Locate the incident in the “In Progress” column (only incidents locked by the current operator show the action buttons).
2. Click the “Release” button (× Release).
3. **Expected outcome:** an info toast appears (“Lock rilasciato.”). The incident moves back from “In Progress” to its original column (“Open” or “Escalated”, depending on its underlying status). The incident is immediately claimable by any operator. On all other clients, the card moves back accordingly.

7.6 Resolving an Incident

Only the operator currently holding the claim on an incident can resolve it.

1. Locate the incident in the “In Progress” column (the “Resolve” button is only visible to the lock holder).
2. Click the “Resolve” button (Resolve). The button is disabled if the incident is already resolved.
3. **Expected outcome (success):** a success toast appears (“Ticket risolto correttamente!”). The incident moves to the “Resolved” column on all clients. Its title is displayed with a strikethrough, and the card shows “Solved by: <your username>”. The “Resolved Tickets” KPI increments by 1; the “Open Tickets” (or “Escalated Tickets”) KPI decrements accordingly. The incident can no longer be claimed or resolved again.
4. **Expected outcome (conflict):** if the incident was concurrently modified by the background escalation task between the time the operator claimed it and the resolution attempt (a rare but possible race), the resolution is rejected with an error toast (“Conflitto o errore: Conflitto! Il documento è stato già modificato o non esiste.”). The operator should re-claim the incident and try again.

7.7 Observing Automatic Escalation

Escalation is fully automatic and requires no operator action. However, it is important for operators to understand when and why it happens.

- Any incident that remains in **open** status for more than **5 minutes** is automatically transitioned to **escalated** by a background task running on the server.
- When this happens, the incident moves from the “Open” column to the “Escalated” column on every connected client. A prominent warning toast appears on all clients: “Attenzione: Il ticket <title> ha superato i limiti SLA ed è stato ESCALATO!” (displayed for 8 seconds, longer than the default 5 seconds).
- Escalated incidents can still be claimed and resolved through the same workflow described above. Claiming an escalated incident moves it to “In Progress”.
- The “Escalated Tickets” KPI counter reflects the current number of unclaimed escalated incidents.

7.8 Using Filters and Search

The filters in the horizontal bar between the chart row and the Kanban board allow operators to narrow the visible incidents:

Text search. Type any substring in the search field. Only incidents whose title contains the substring (case-insensitive) are displayed in the Kanban columns. Clearing the field restores all incidents.

Escalated-only toggle. Click the “Mostra solo Escalated” button. When active (highlighted in orange), only incidents with status **escalated** are shown across all columns. Click again to deactivate.

My-tickets toggle. Click the “I Miei Ticket” button. When active (highlighted in blue), only incidents where the current operator is the creator (**createdBy**), the current lock holder (**lockedBy**), or the resolver (**closedBy**) are shown. Click again to deactivate.

Important: the KPI counters in Row 1 (Total, Open, Escalated, Resolved, Users On-line) are **never affected by filters**. They always show the true global system state. Only the Kanban columns and their per-column count badges change in response to filters.

7.9 Notifications

The system uses non-blocking **toast notifications** (bottom-right corner, auto-dismiss after 5 seconds unless hovered) to inform the operator of events:

- **Info (blue):** a new incident was created by another operator (“Nuovo ticket registrato: <title>”).
- **Success (green):** the operator’s own action succeeded (creation, claim, resolution); or another operator resolved an incident (“Il ticket <title> è stato risolto da <name>”).
- **Warning (orange):** an incident was automatically escalated (“Attenzione: Il ticket <title> ha superato i limiti SLA ed è stato ESCALATO!”). This notification is displayed for 8 seconds.
- **Error (red):** an action failed — claim conflict (“Impossibile prendere in carico: ...”), release conflict (“Impossibile rilasciare: ...”), resolution conflict (“Conflitto o errore: ...”), or a network error (“Errore di rete durante ...”).

Notifications can be dismissed early by clicking on them. They can also be dragged horizontally to dismiss. A progress bar at the bottom of each notification shows the remaining display time.

7.10 Multi-Operator Collaboration

The system is designed for simultaneous use by multiple operators. To experience the collaborative features:

1. Open `http://board.local` in two or more browser windows (or on different machines on the same network).
2. Log in with different usernames in each window.
3. Perform actions (create, claim, resolve) from any window and observe the real-time synchronization on all other windows:
 - Incident cards appear, move between columns, and disappear simultaneously across all clients.
 - The “Users Online” KPI reflects the number of distinct connected operators.
 - Attempting to claim an incident already held by another operator produces an immediate conflict notification.
4. Close a browser window and observe the “Users Online” count decrease within a few seconds.

7.11 Connection Resilience

The application handles connection interruptions transparently:

- If the WebSocket connection is lost (e.g., due to a network glitch, a pod restart, or a rolling update), the Socket.IO client automatically attempts to reconnect with exponential backoff.

- Upon reconnection, the client fetches the full, current incident list from the server — including which incidents are currently claimed and by whom — ensuring the displayed state is never stale, and never partially restored.
- The server can also *request* a resynchronization: if a pod detects that it has missed a segment of the database change feed and cannot replay it, it tells its own clients to reload the board rather than letting them drift. This happens without any operator-visible interruption.
- The “Live Sync Active” indicator (green pulsing dot) in the dashboard header confirms that the real-time connection is operational. If the connection is lost, the Socket.IO client’s reconnection logic will restore it without manual intervention.
- No data is lost during a disconnection: all mutations are sent via HTTP (not WebSocket), so they either succeed and are persisted in MongoDB or fail and produce an error notification. The real-time event stream is used only for *receiving* updates, not for sending commands.

8 Self-evaluation

- An individual section is required for each member of the group
- Each member must self-evaluate their work, listing the strengths and weaknesses of the product
- Each member must describe their role within the group as objectively as possible.

It should be noted that each student is only responsible for their own section.

9 Future works

The current iteration deliberately scopes out several directions in order to keep the focus on the distributed-systems aspects of the project. The most relevant candidates for future work are:

- **Transport security.** Traffic between the browser and the Ingress, and between the `api` pods and their backing services (MongoDB, Redis), is currently unencrypted (section 2.4). A production-grade deployment would terminate TLS at the Ingress and enable TLS for the MongoDB Replica Set and Redis connections, plus encryption at rest for the MongoDB volumes.
- **Fine-grained authorization.** All authenticated operators currently share a single implicit role. Introducing roles (e.g., operator vs. supervisor) with differentiated permissions — such as restricting who can resolve escalated incidents, or exposing an audit trail of who claimed/released/resolved which incident — would be a natural extension of the authorization model described in section 4.4.

- **Account lifecycle hardening.** Login currently has no rate limiting, account lockout, or password-reset flow. Adding these (e.g., exponential backoff per username/IP, a lockout after repeated failures) would close an obvious brute-force gap without changing the stateless nature of the `api` tier.
- **Robust error handling on the client.** As noted in section 5.2 (FR10), the incident-creation request path does not check the HTTP response status or catch network failures, so a failed `POST /api/incidents` still shows a success toast. The fix is mechanical (wrap the `fetch` in a `try/catch`, branch on `response.ok`, and surface a failure toast) but was left out of the current iteration since it does not affect the distributed coordination logic under test.
- **Responsive / mobile layout.** The dashboard currently uses a fixed-column CSS Grid layout designed for a desktop viewport, with no `@media` breakpoints. Supporting narrower viewports would require reflowing the KPI row and Kanban columns (e.g., collapsing to a single-column, tab-based view below a given width) rather than a purely cosmetic change.
- **Drag-and-drop interactions.** Kanban cards currently move between columns automatically as a side effect of status/lock changes; there is no direct manipulation. Adding drag-and-drop (e.g., dragging a card into “In Progress” to claim it) would need to reconcile the optimistic client-side move with the same claim/release/resolve endpoints and lock semantics already in place, rather than introduce a separate mutation path.
- **Horizontal scaling of stateful services.** MongoDB already runs as a 3-node Replica Set and `api` scales horizontally via the HPA, but Redis remains a single instance (section 3.7) and is therefore both a scalability ceiling and the one component whose loss makes the whole system unavailable. The degradation is not partial: every `api` readiness probe pings Redis, so while Redis is down *all* replicas fail readiness at once and leave the Service together, and the board stops serving reads as well as writes. No durable data is at risk — incidents and credentials live in MongoDB — but the coordination state is: Redis runs without persistence, so a restart silently drops every active claim and the current leadership, and operators must claim their tickets again. Moving to Redis Sentinel or Redis Cluster would remove this single point of failure at the cost of additional operational complexity. A cheaper intermediate step would be a degraded read-only mode, which does *not* exist today: it would require both dropping Redis from the readiness probe and letting `GET /api/incidents` return the snapshot without lock information when `READ_LOCKS_SCRIPT` cannot run, while claims, resolutions and cross-pod broadcasts remain unavailable.